

Grado en Ingeniería Software

Tema 2
Estilos y Patrones de Diseño Software

Índice de la Presentación

- Introducción
- Estilos Arquitectónicos y patrones de Diseño
- Vistas arquitectónicas y diseño con UML
- Otros patrones de diseño
- Conclusiones
- Temas complementarios: Notación C4

Introducción

- Las arquitecturas software utilizan un conjunto de estilos arquitectónicos y patrones de diseño software para construirlas
- Son elementos de conocimiento reutilizable
- Las AS son heterogéneas, los estilos y patrones se suelen combinar
- Básicamente, la diferencia entre estilos y patrones reside en el tamaño (los estilos son mas grandes en términos de clases y subsistemas que los patrones)
- Los estilos son una de las primeras decisiones de diseño a considerar

Introducción

- Los estilos y los patrones determinan la forma de la arquitectura
- Diferentes estilos sirven para diferentes propósitos





Patrones y Estilos

- Un estilo arquitectónico de diferencia de un patrón de diseño en el tamaño (granularidad diferente).
- Los estilos involucran subsistemas mientras que los patrones son un conjunto reducido de clases.
- Ambos sirven para el mismo propósito que es organizar los elementos de una AS.
- Evitan reinventar soluciones de diseño ya probadas.
- **En diseño software, se debe seleccionar primero el estilo o estilos (si hubiera más de 1) y luego los patrones (caso de que se puedan aplicar).**
- Si no hubiera un estilo definido la solución de diseño sería un conjunto de clases

Algunos Estilos Arquitectónicos
(Garlan & Shaw, 1995; Clements et al., 1998-2003)

- Tuberías y filtros (Pipes and filters)
- Estilo por capas (Layers)
- Cliente-servidor (Client-Server)
- Pizarra (blackboard)
- Modelo-Vista-Controlador (MVC)
- Máquina virtual
- Dirigido por eventos (Event-driven)
- Conexión a pares (Peer-to-Peer)
- Orientado a servicios (SOA) y Microservicios

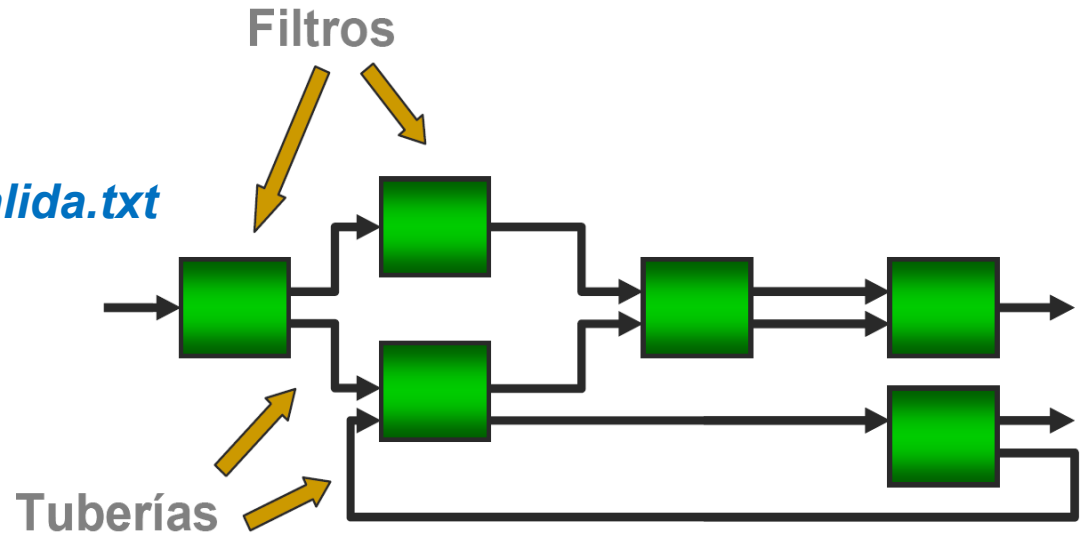
Describen la AS con una granularidad de sus componentes poco detallada.

Estilo Pipe and Filter

*Un **filtro** lee flujos de datos de sus entradas y genera flujos de datos a sus salidas.*

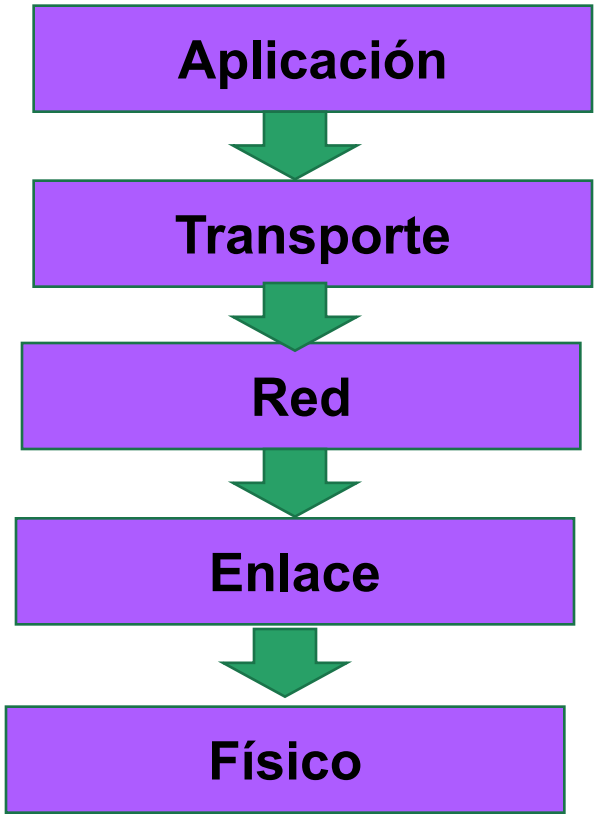
- *El **filtro** transforma los datos incrementalmente (se comienza a generar salida antes de acabar de leer toda la entrada).*
- *La **tubería** transmite el flujo de datos.*

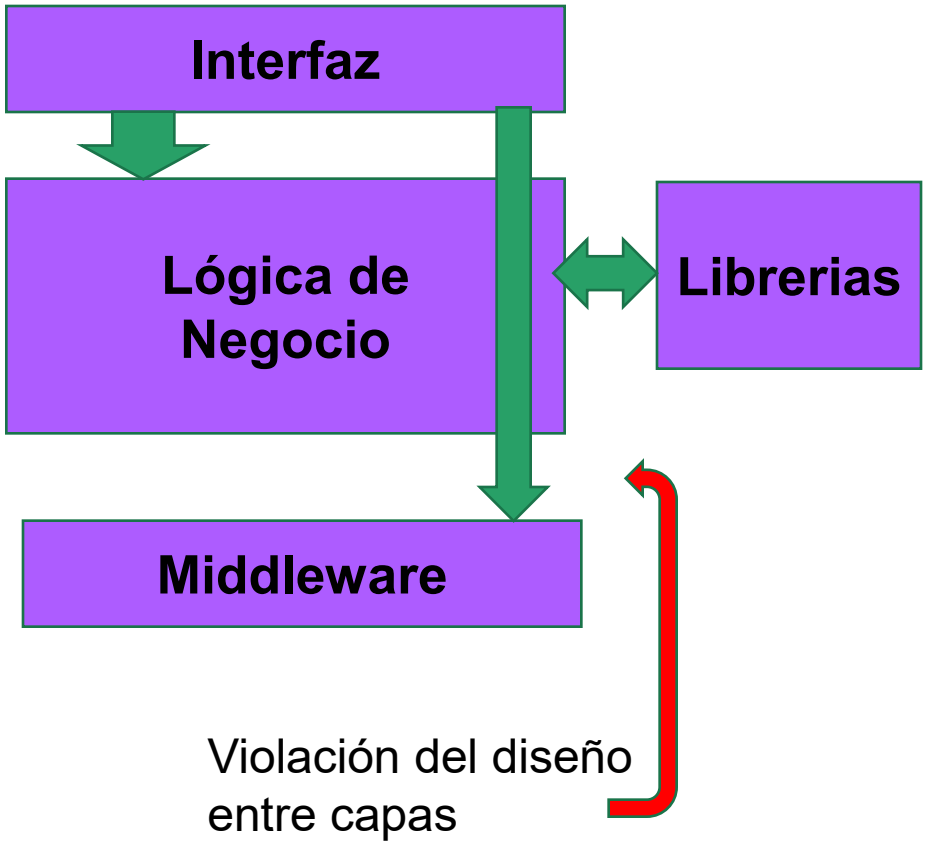
Comando Unix
ls –al | grep "fichero" > salida.txt



Estilos por Capas

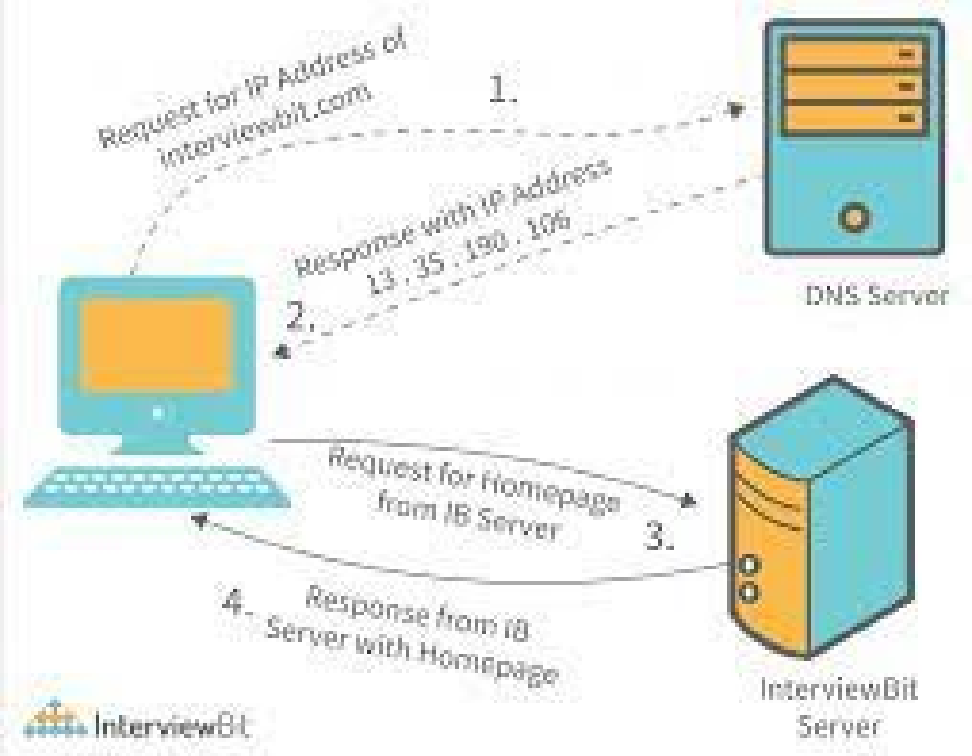
- Cada capa soporta un API (servicios) para que lo utilice la capa superior. A veces las capas no están claras y hay llamadas directas entre elementos de una capa (pero violan los principios de diseño). Podemos encontrar capas horizontalmente.





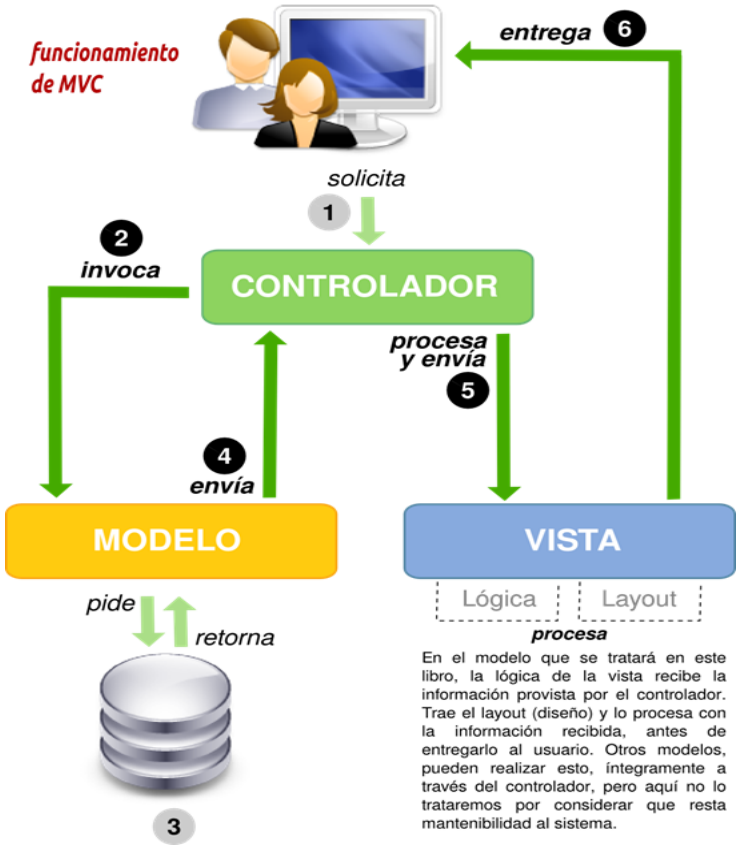
Estilo Cliente-Servidor

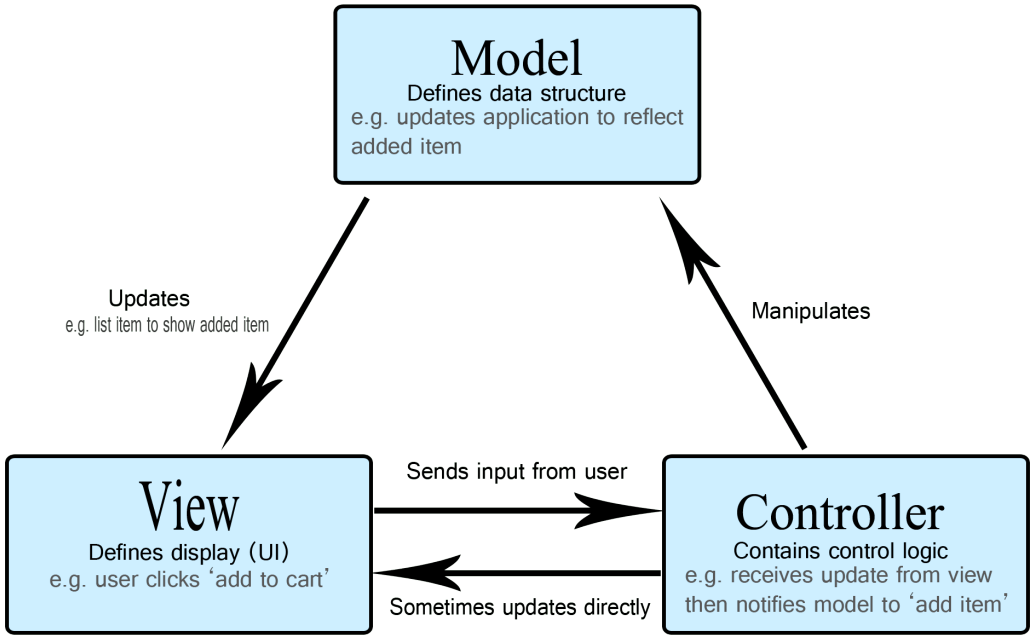
- Modelo utilizado en Internet
- Los clientes acceden a datos y aplicaciones en los servidores
- C/S distribuidos (redes peer-to-peer)
- Peticiones asíncronas y respuestas a eventos



Estilos MVC

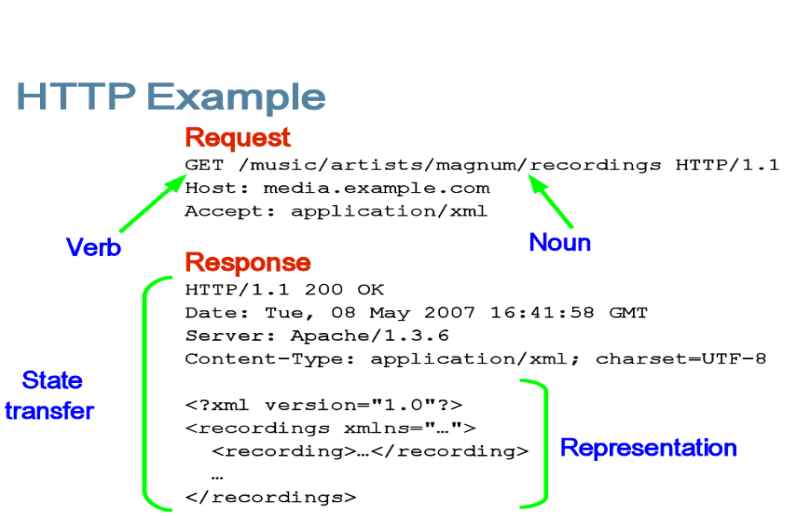
- Estilo para sistemas interactivos (desde años ‘80)
- Diferentes vistas de los mismos datos
- Posibilidad de interacción multimodal

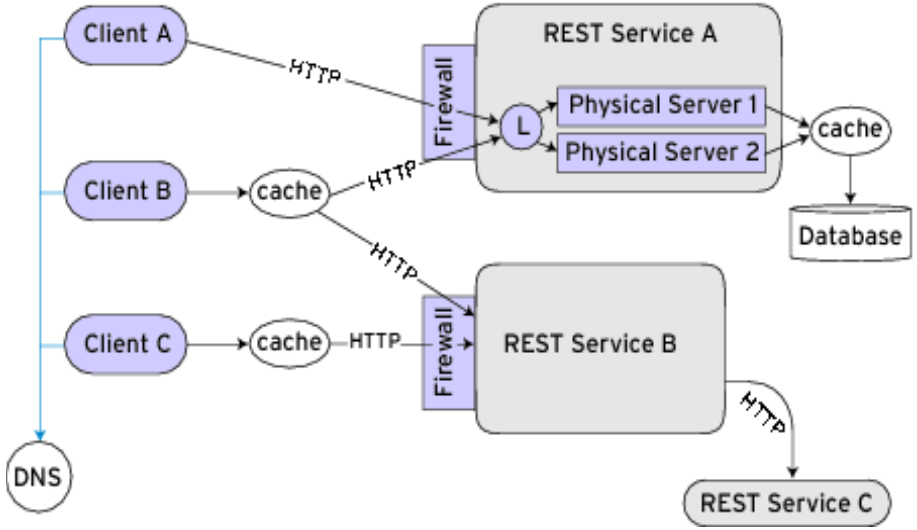




Estilo REST

- REST (Representational State Transfer)
- Utilizado en para acceso a servicios Web y desarrollo Web
- Es una arquitectura cliente-servidor sin estado (stateless). El cliente almacena el estado
- Proporciona una interfaz uniforme
- Es cacheable





Estilo RESTful

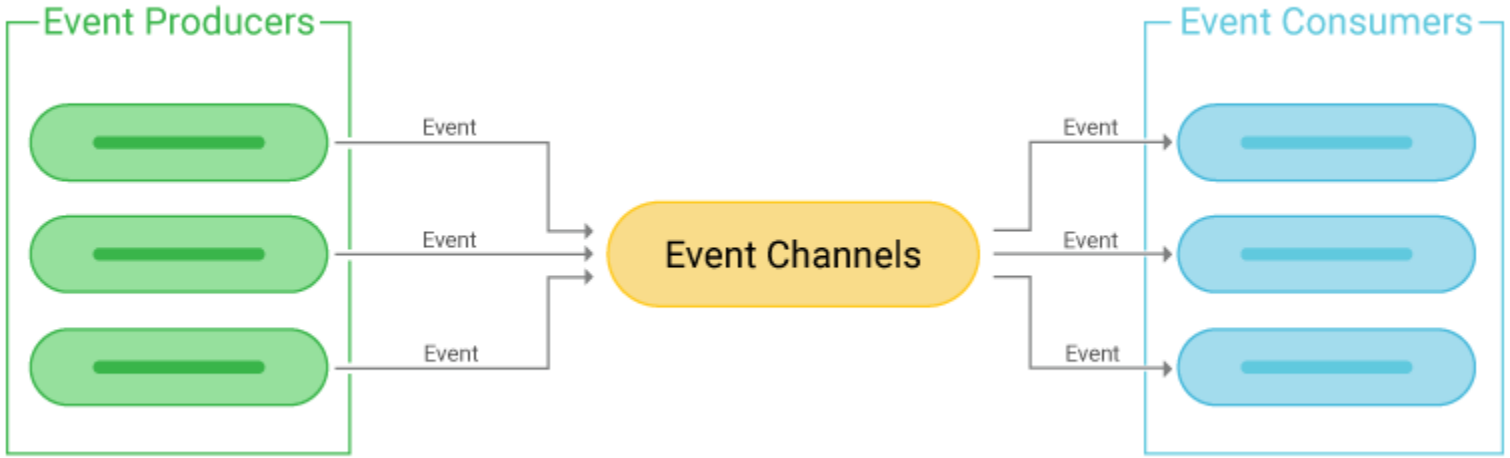
El estilo RESTful se basa en 6 principios: *C/S, Statelessness, Cacheability, Layered system, Uniform interface, code on demand (optional)*

Desde el punto de vista de la arquitectura de mejora los siguientes atributos de calidad:

- **Performance**: Las interacciones entre las páginas Web y servidores mejoran las prestaciones
- **Scalability**: para soportar un mayor de número de componentes e interacciones entre ellos
- **Simplicity**: de la interfaz mediante URIs
- **Modifiability**: de los componentes que cambian (incluso si la aplicación está en ejecución)
- **Reliability**: mejora de la resistencia a fallos
- **Portability**: cuando se mueven componentes de código con sus datos

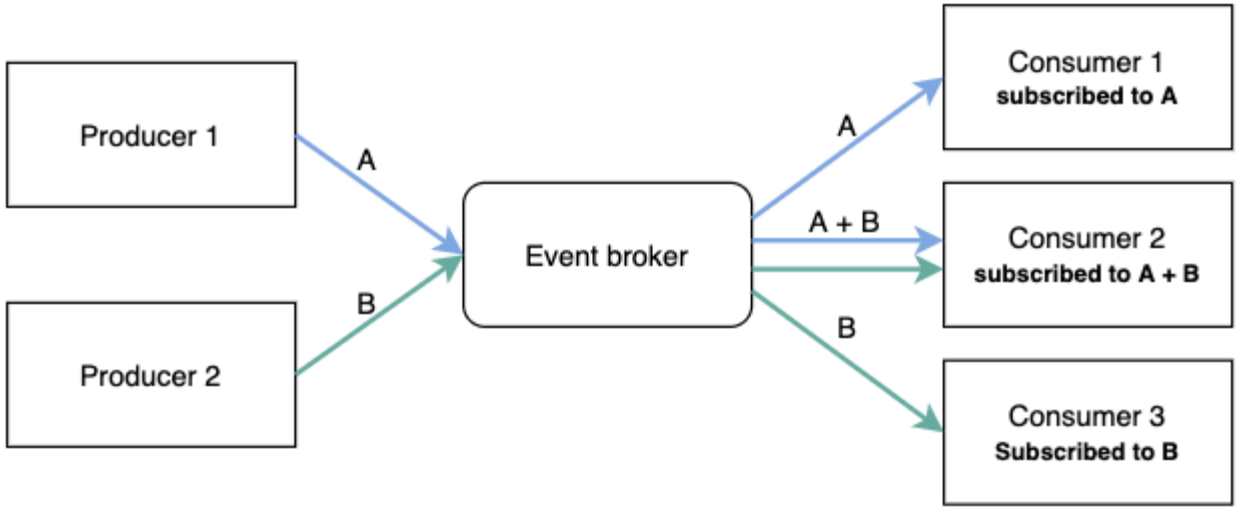
Estilos por Eventos

- Las arquitecturas dirigidas por eventos (EDA Event-driven Architectures) se basan en la detección, consumo y reacción a eventos
- Un evento representa un cambio significativo en el estado de un sistema
- Los eventos se transmiten entre sistema poco acoplados (“loosely coupled”) mediante mensajes



Estilos por Eventos

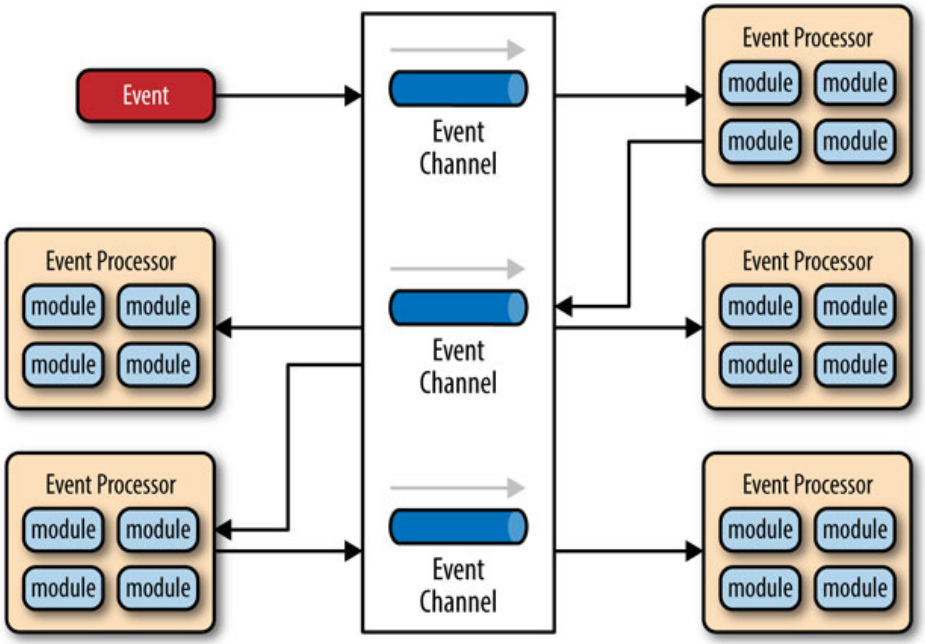
- Una arquitectura EDA tiene emisores de eventos, consumidores y canales para transmitir los eventos
- Los emisores no conocen a los consumidores de los eventos
- Los consumidores son responsables de reaccionar a los eventos
- Java Swing es un EDA (ActionListener, ActionEvent)
- Existe un módulo encargado de la gestión de los eventos
- Monitorizan los datos del entorno y analizan los eventos para proporcionar la respuesta más adecuada
- Utilizan el patrón "**publish-subscribe**" para intercambio de mensajes

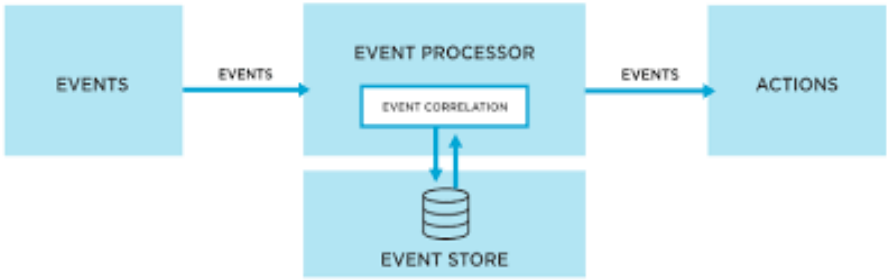


Estilos por Eventos

Diferentes estilos o formas de procesar un evento:

- **Eventos simples**: relacionados con cambios medibles en una condición (e.g., un sensor que detecta temperatura)
- **Eventos en "stream":** son eventos que se envían en cadena a suscriptores
- **Eventos complejos**: representan confluencia de eventos y requieren intérpretes de eventos sofisticados





UML: Diagramas Paquetes

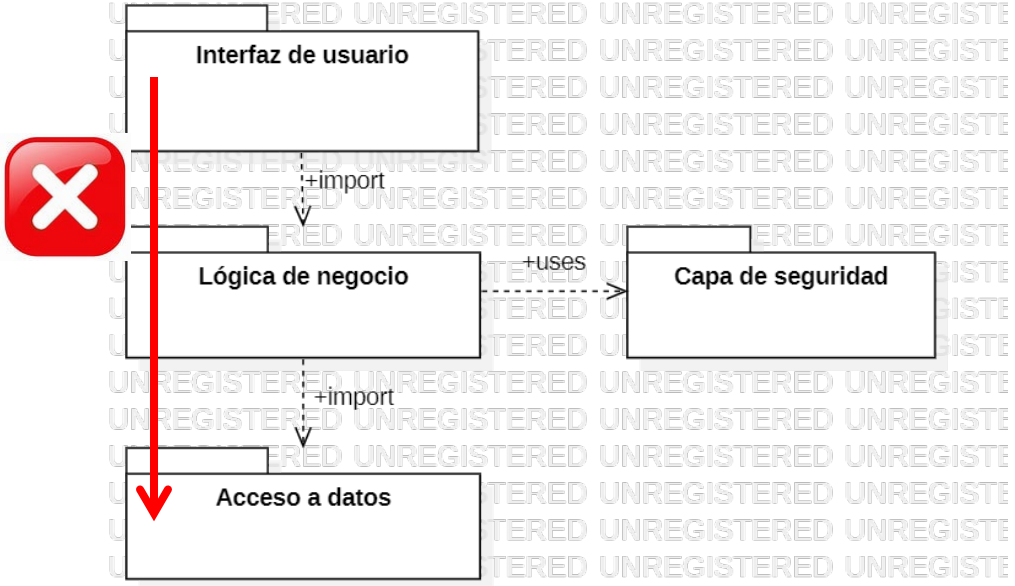
- Describen subsistemas
- No debe haber paquetes vacíos
- Normalmente contienen clases u otros paquetes
- Pueden incluir componentes UML
- Los paquetes deben llevar un estereotipo (“<<stereotype>>”) que indique la misión del paquete o subsistema
- Los paquetes definen el estilo o estilos de una arquitectura software



UNREGISTERED UNREGISTERED

UML: Diagramas Paquetes

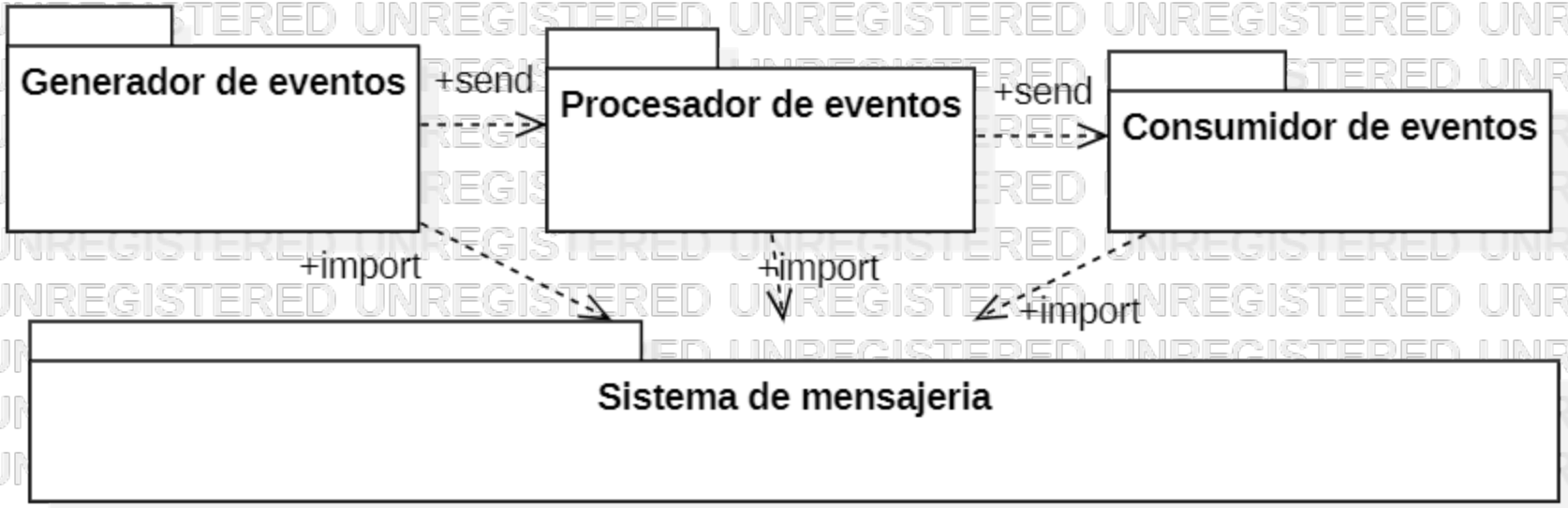
- Estilo por capas en UML
- Son capas verticales (“layers”)
- Usan la relación de <<import>> entre paquetes
- Un paquete de nivel superior importa los servicios de la capa inferior
- Las relaciones no deben atravesar capas
- Cada subsistema describe un nivel de responsabilidad
- Pueden existir paquetes al mismo nivel de responsabilidad



UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED

UML: Diagramas Paquetes

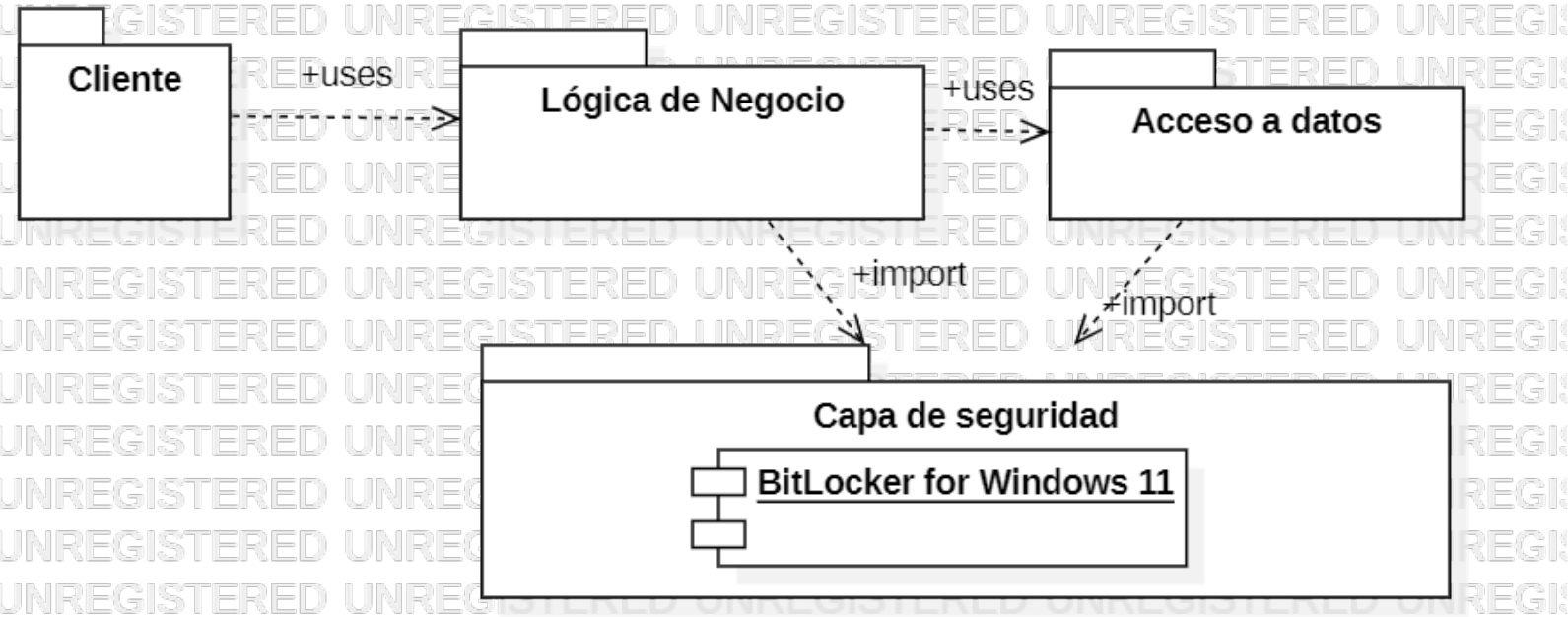
- Estilo por eventos
- Tres paquetes en horizontal
- El paquete inferior permite la comunicación “publish-subscribe”
- Cada paquete define su nivel de responsabilidad



UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED

UML: Diagramas Paquetes

- Estilo Cliente/Servidor
- Tres capas en horizontal (“tiers”)
- Pueden existir capas de nivel inferior
- Un paquete puede contener componentes UML

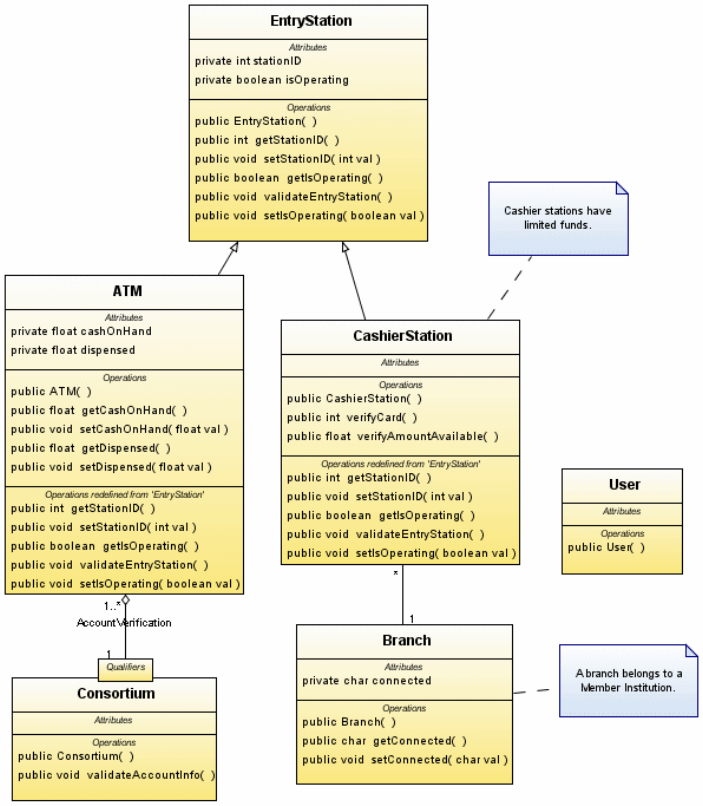


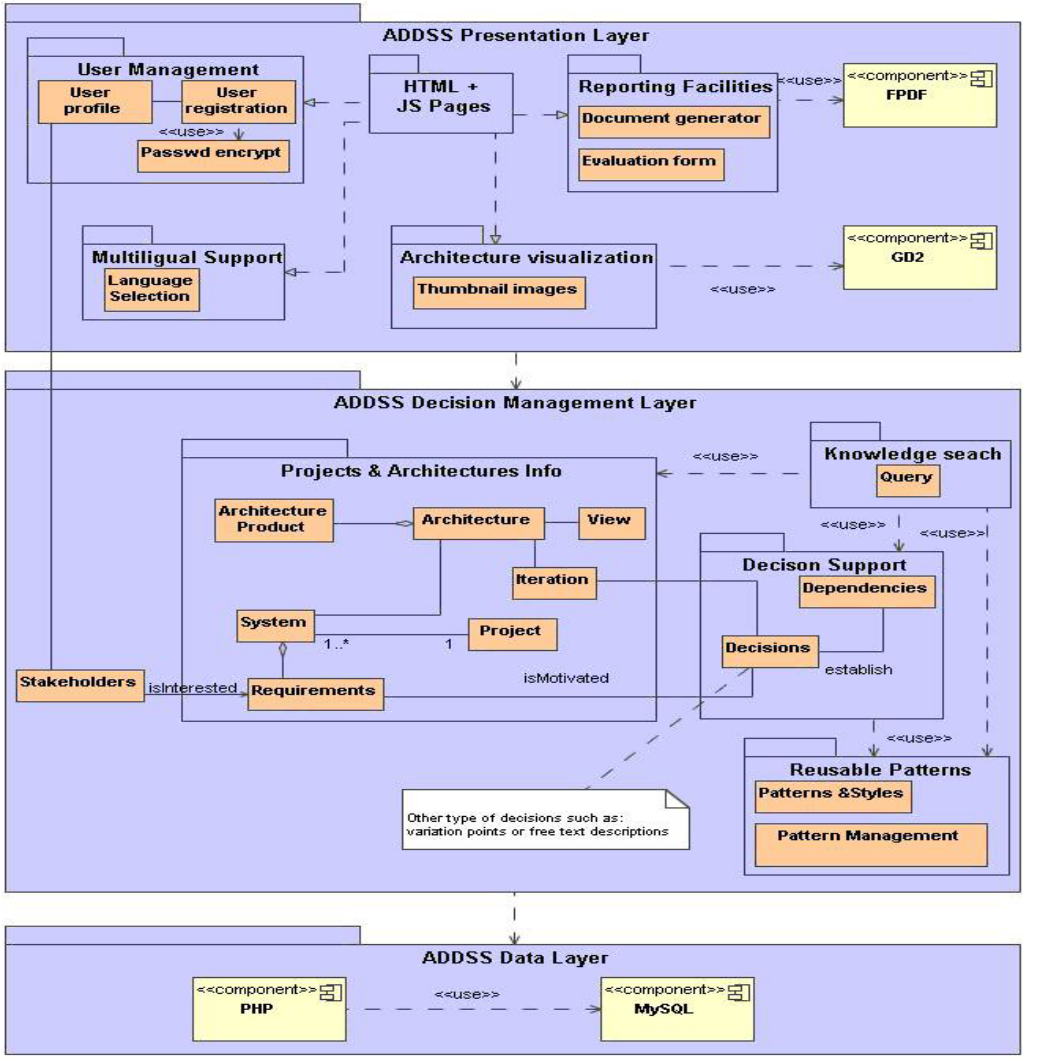
UNREGISTERED

Arquitecturas y Diseño UML

Ejemplo de vista Estática

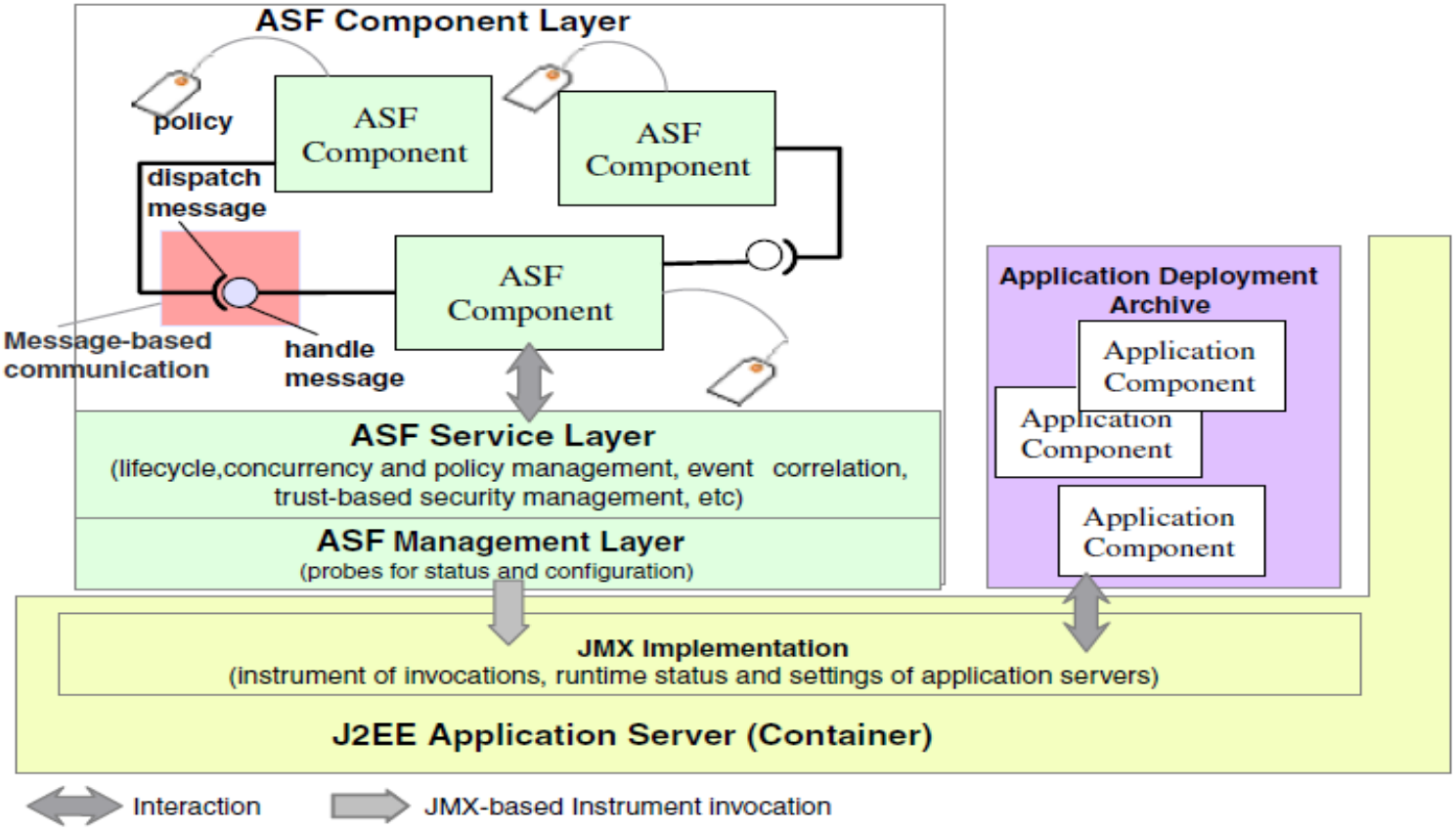
Diagrama clases y paquetes





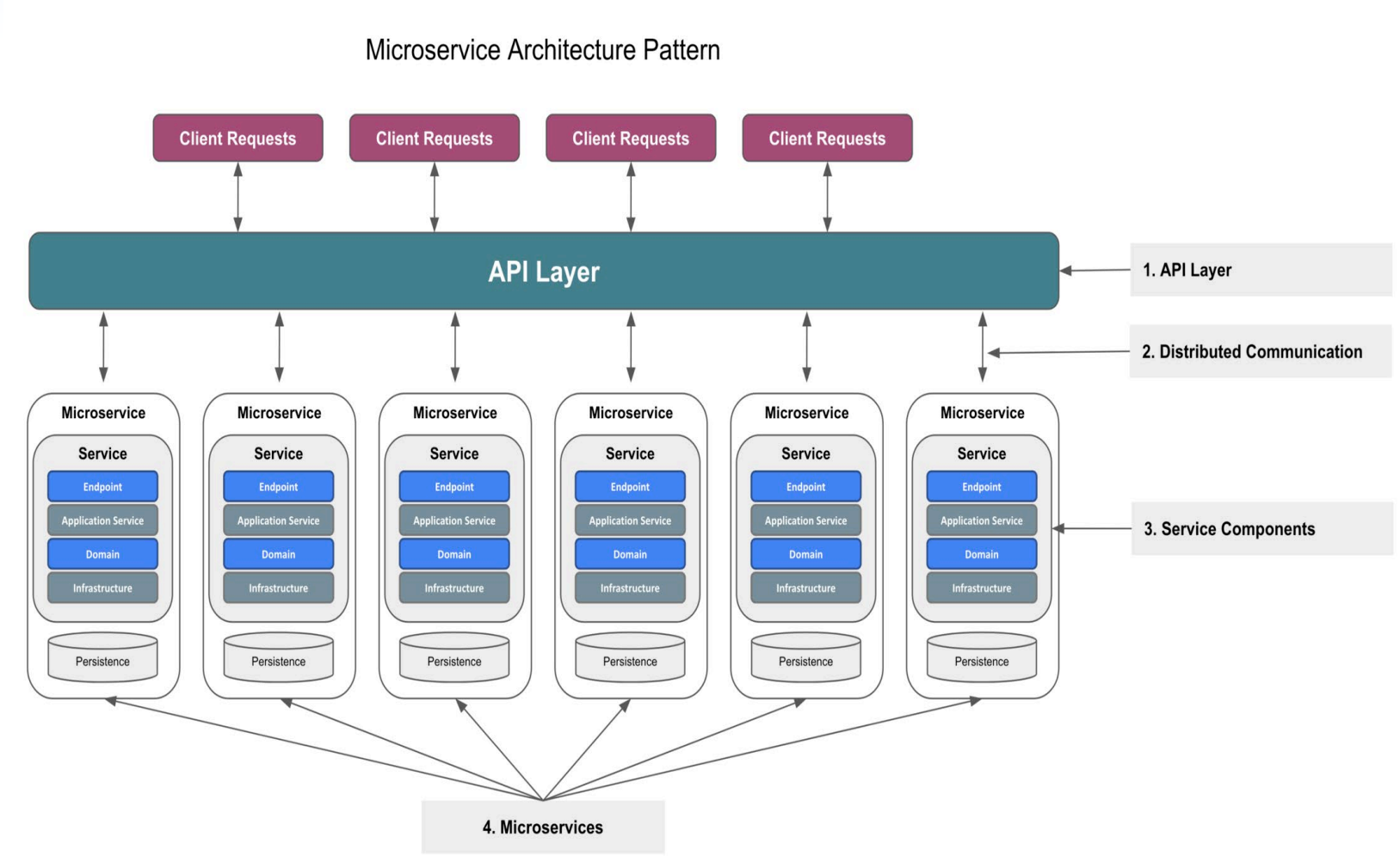
Ejemplos de Arquitecturas

Arquitectura por Capas (ASF: Adaptive Server Framework)



Aquí se muestran 3 capas verticales y una de ellas tiene una capa horizontal añadida en color morado

Ejemplos de Arquitecturas



Patrones de Diseño

- La construcción de arquitecturas software se basan en los patrones de diseño y estilos arquitectónicos ya definidos.
- Un **Patrón de Diseño Software** (“design pattern”) es una solución de diseño para un problema recurrente en un contexto determinado.
- Son soluciones de diseño reutilizables ya probadas.

Patrones y Estilos

- Cronología e historia de los patrones software
  - The Timeless Way of Building (C. Alexander, 1979).
  - Lenguajes de patrones para programas OO (Cunningham y Beck, 1987).
  - Patrones de diseño (Gamma et al., 1995, GoF book).
  - Estilos arquitectónicos (Buschmann et al., 1996-2000).
  - Patrones de arquitecturas de aplicaciones empresariales (M. Fowler, 2002).
  - Patrones actuales centrados en tecnologías y plataformas (e.g.: J2EE, Web. Microservicios).

Categorías de Patrones

Existen diversos tipos de patrones de diseño

- **Creacionales:** Creación de objetos
- **Estructurales:** Estructura de las clases
- **Comportamiento:** Aspectos dinámicos del sistema
- **Concurrencia:** Scheduler, Lock, Thread Pool
- **Interacción (problemas de HCI)**
- **Workflow & process automation**
- **Microservicios**
- **IA generativa**

Patrones de Diseño

- Diferentes categorías de patrones (creacionales, comportamiento, estructura)
- Mas de 40 tipos de patrones de diseño
- Existen patrones específicos para soluciones empresariales (basados en Java)
- Los más recientes son los patrones para arquitecturas de micro servicios y de IA generativa

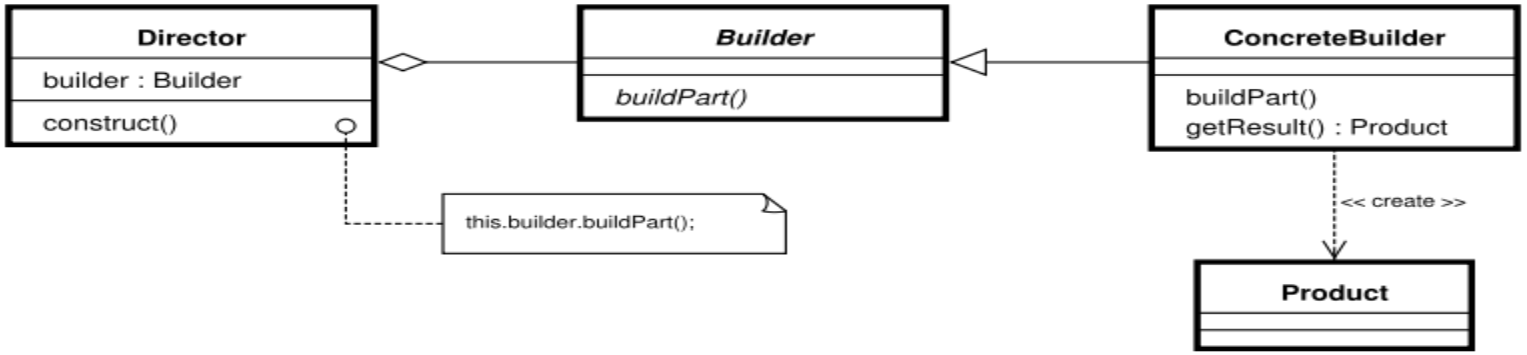
Patrones de Diseño Clásicos

- **Patrones creacionales**: patrones de diseño software que solucionan problemas de creación de instancias
- **Patrones estructurales**: solucionan problemas de composición (agregación) de clases y objetos
- **Patrones de comportamiento**: ofrecen soluciones respecto a la interacción y responsabilidades entre clases y objetos

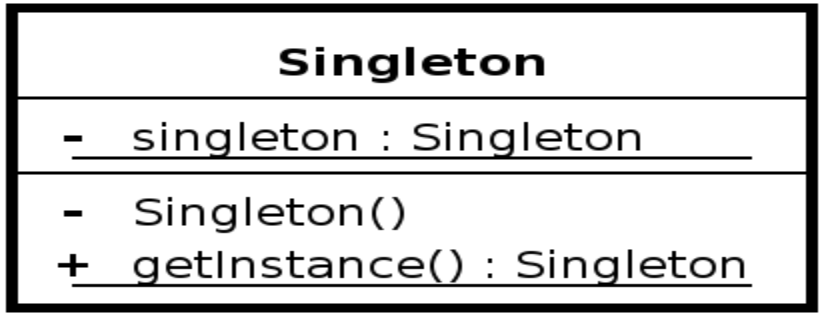
Esta es la clasificación clásica de patrones de diseño del libro de Gamma (GoF) aunque existen otras clasificaciones.

Patrones Creacionales

- **Builder:** abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto

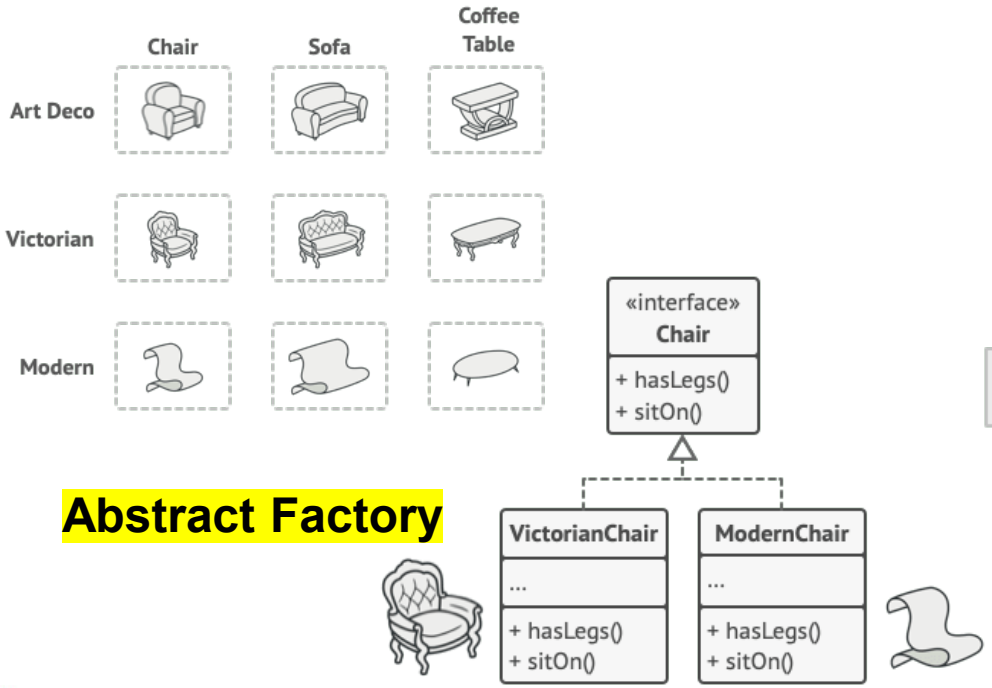


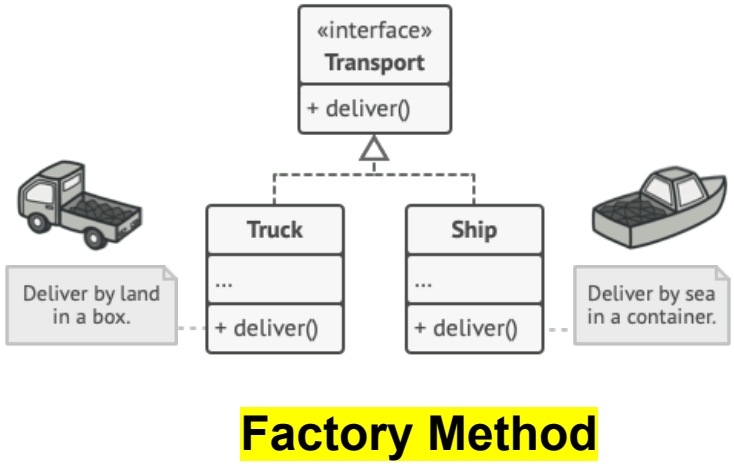
- **Singleton:** garantiza la existencia de una única instancia para una clase. Permite restringir la creación de objetos a una clase



Patrones Creacionales

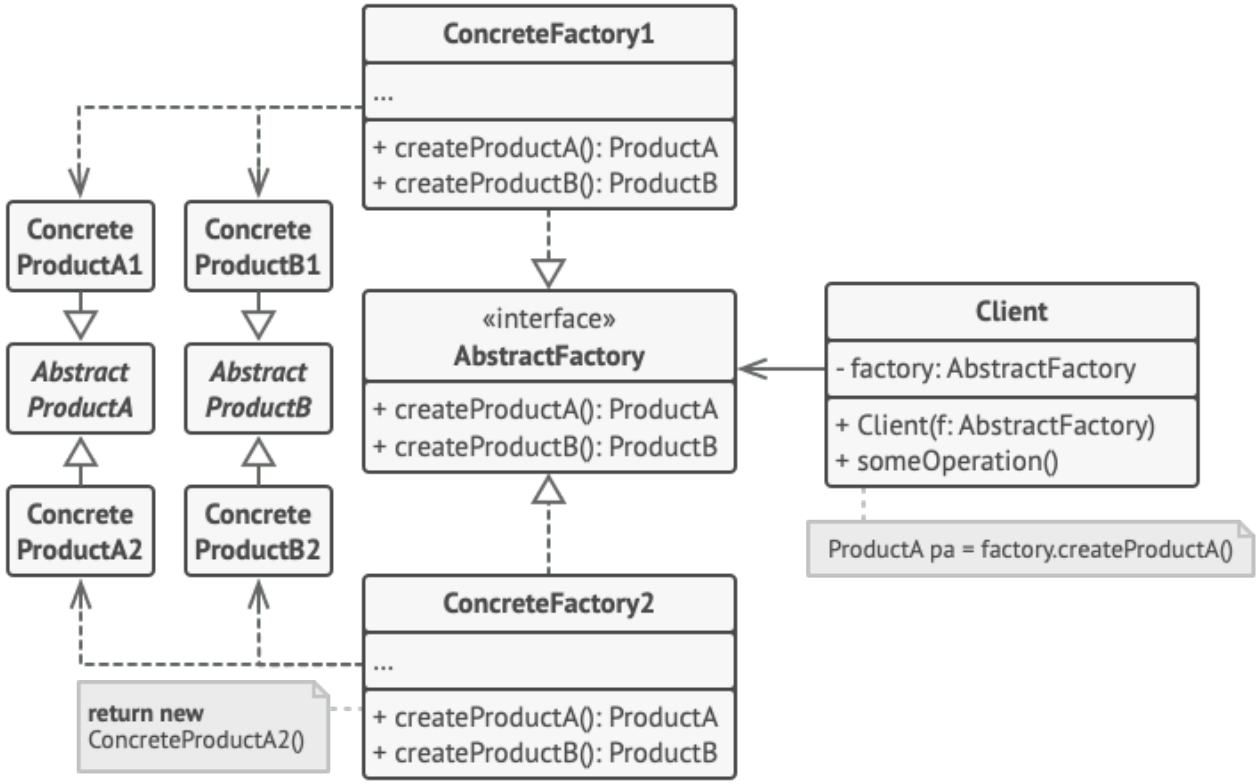
- **Abstract Factory:** permite trabajar con objetos de familias relacionadas y haciendo transparente el tipo de familia que se esté utilizando
- **Factory Method:** centraliza en una clase constructora la creación de objetos de un subtipo determinado, ocultando al usuario la diversidad de casos particulares. Permite a las subclases modificar el "tipo" del objeto a crear. Es una simplificación del *Abstract Factory*





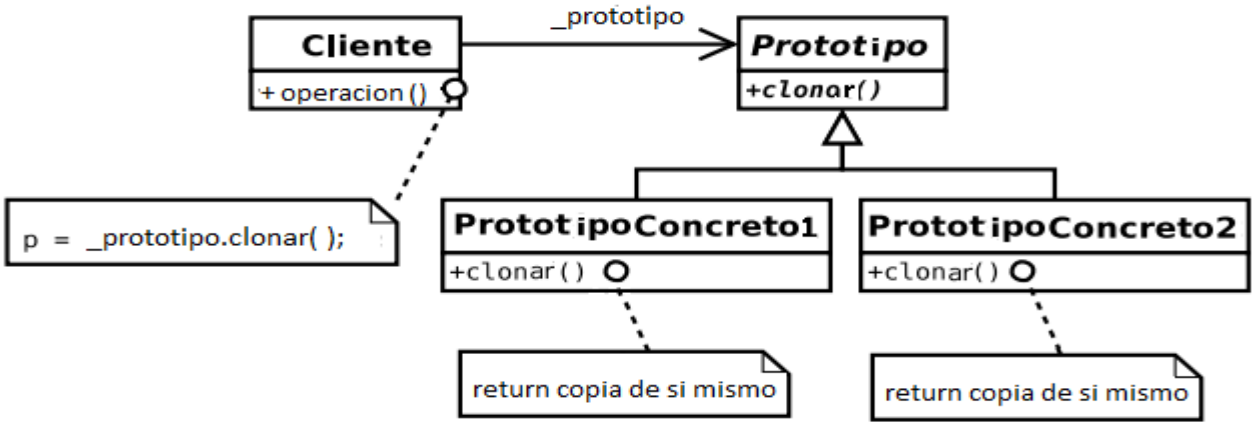
Patrón Abstract Factory

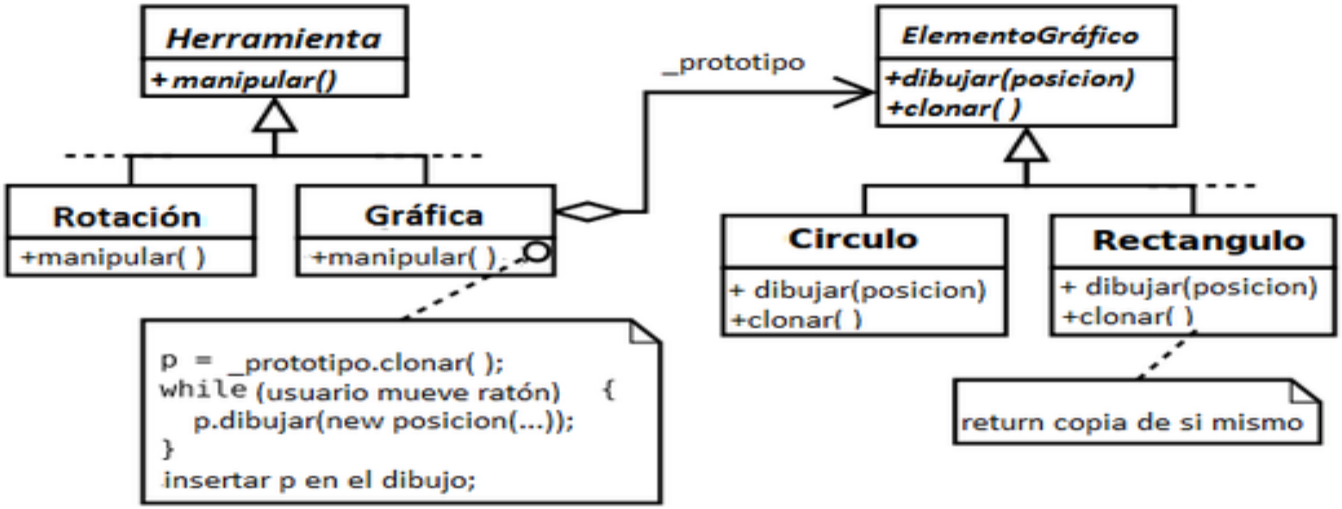
- **Propósito:** Encapsula un grupo de fábricas (“factories”) individuales bajo un tema común y utiliza interfaces genéricos para crear objetos concretos. Una factoria es una interfaz para crear objetos o familias de objetos sin especificar sus clases concretas



Patrón Prototype

Propósito: tiene como finalidad crear nuevos objetos duplicándolos



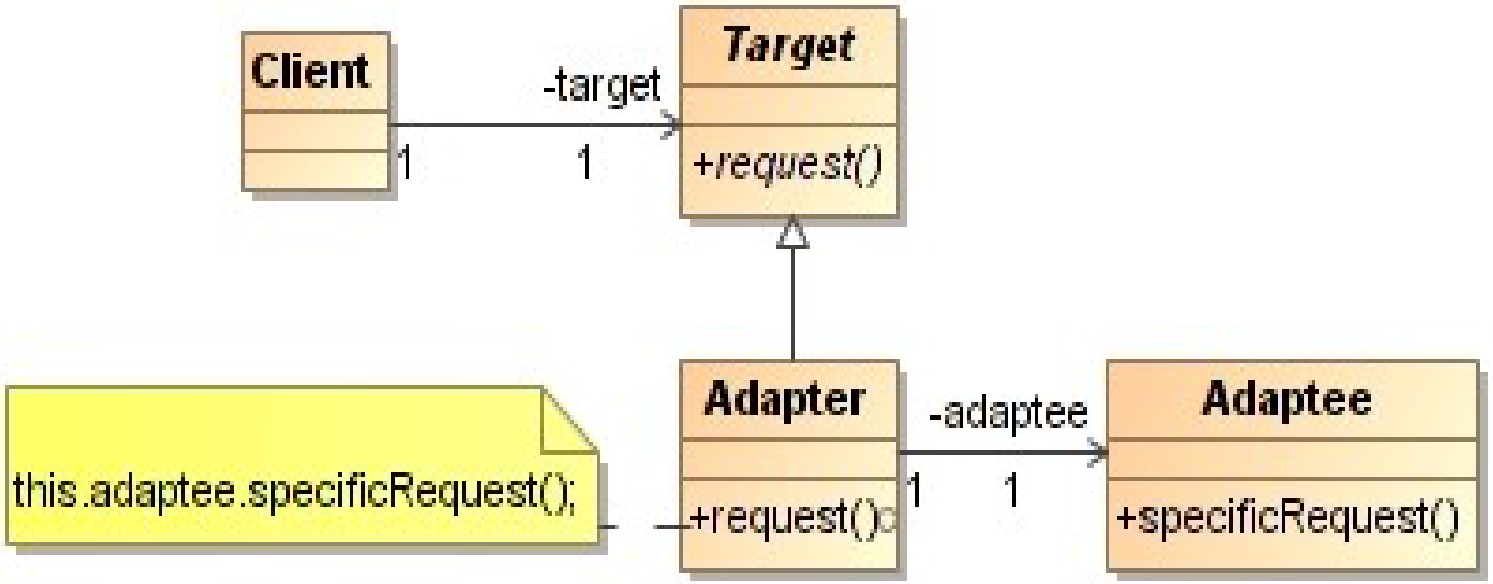


Patrones Estructurales

- **Adapter (wrapper):** Transforma una interfaz en otro. Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla
- **Decorator:** Añade funcionalidad a una clase dinámicamente
- **Facade:** Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema
- **Proxy, Composite, Bridge, etc.**

Patrón Adapter

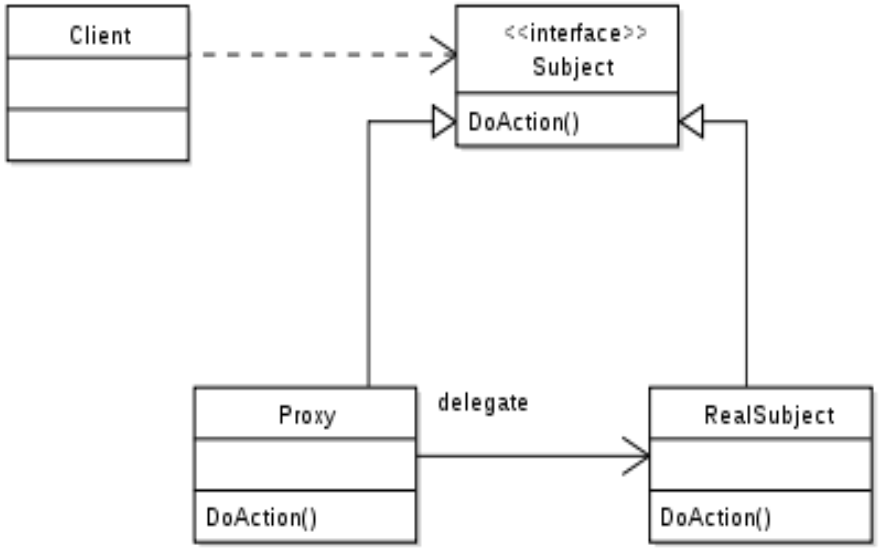
- **Target** define la interfaz específica del dominio que *Client* usa.
- **Client** colabora con la conformación de objetos para la interfaz *Target*.
- **Adaptee** define una interfaz existente que necesita adaptarse
- **Adapter** adapta la interfaz de *Adaptee* a la interfaz *Target*

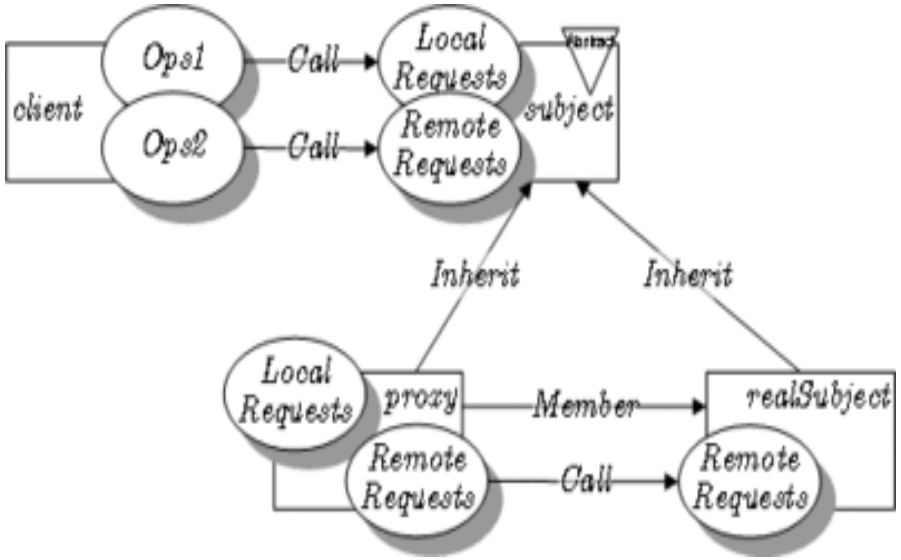


Tipicamente utilizado en programas antiguos para proporcionar un front-end moderno. Por ejemplo, wrappers para programas de sistemas bancarios escritos en COBOL se les proporciona un envoltorio o front-end Web que es el que accede a los datos

Patrón Proxy

Propósito: Proporciona un intermediario de un objeto para controlar su uso. Es una clase funcionado como una interfaz de otro elemento (ej: una conexión de red, un servidor)

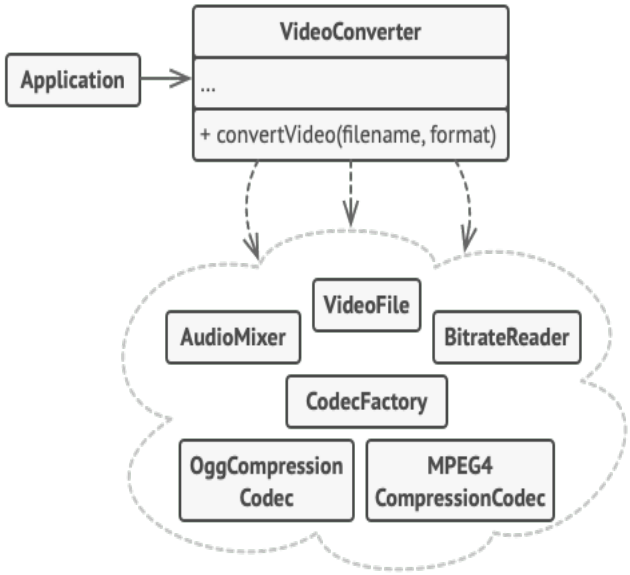


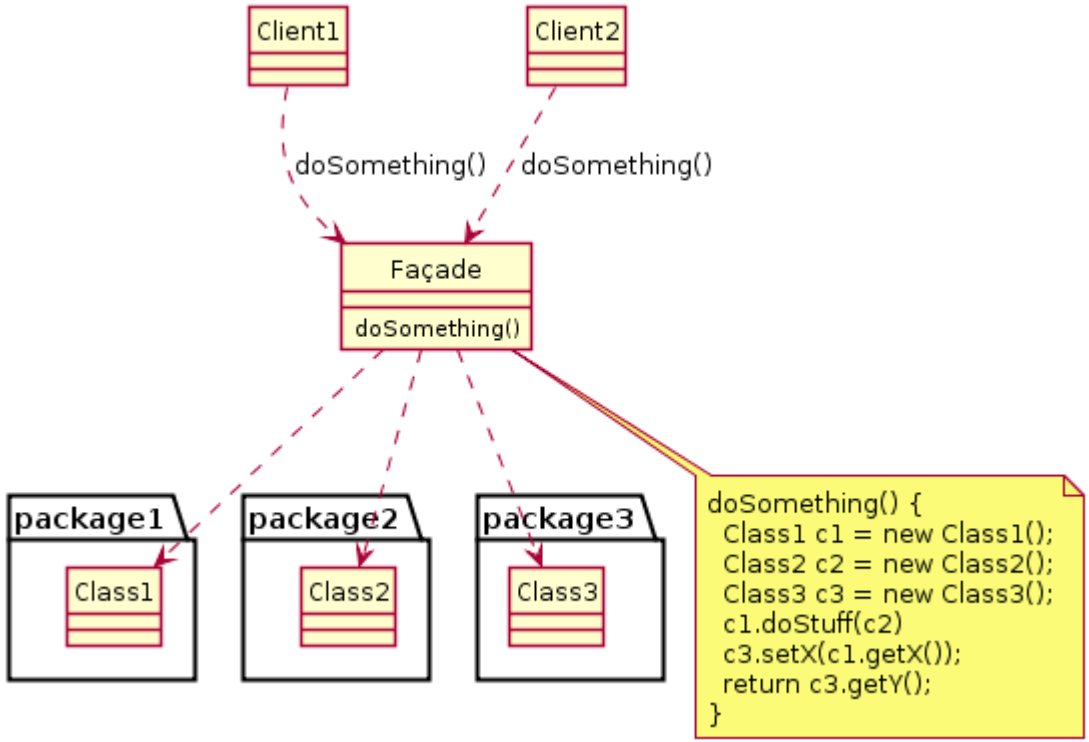


Muy habitual en sistemas Web para hacer la caché de las páginas más frecuentemente visitadas.

Patrón Facade

Propósito: Proporciona una interfaz simplificada a una librería, un framework o un conjunto de clases complejas. Es una alternativa al abstract factory y es similar al patrón Mediator.



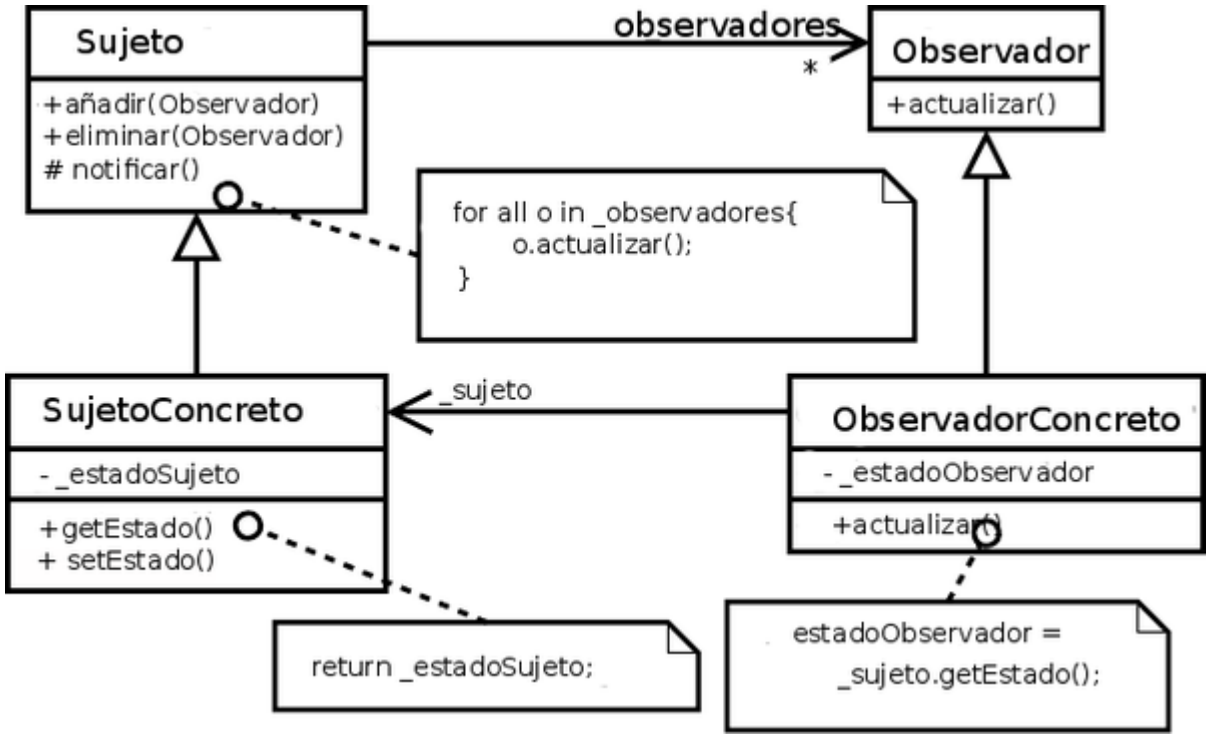


Patrones de Comportamiento

- **Chain of responsibility:** Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada
- **Strategy**: Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución
- **State**: Permite que un objeto modifique su comportamiento cada vez que cambia el estado interno
- **Visitor**: Define nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera
- **Command, Interpreter, Iterator, Observer, Mediator, etc**

Patrón Observer

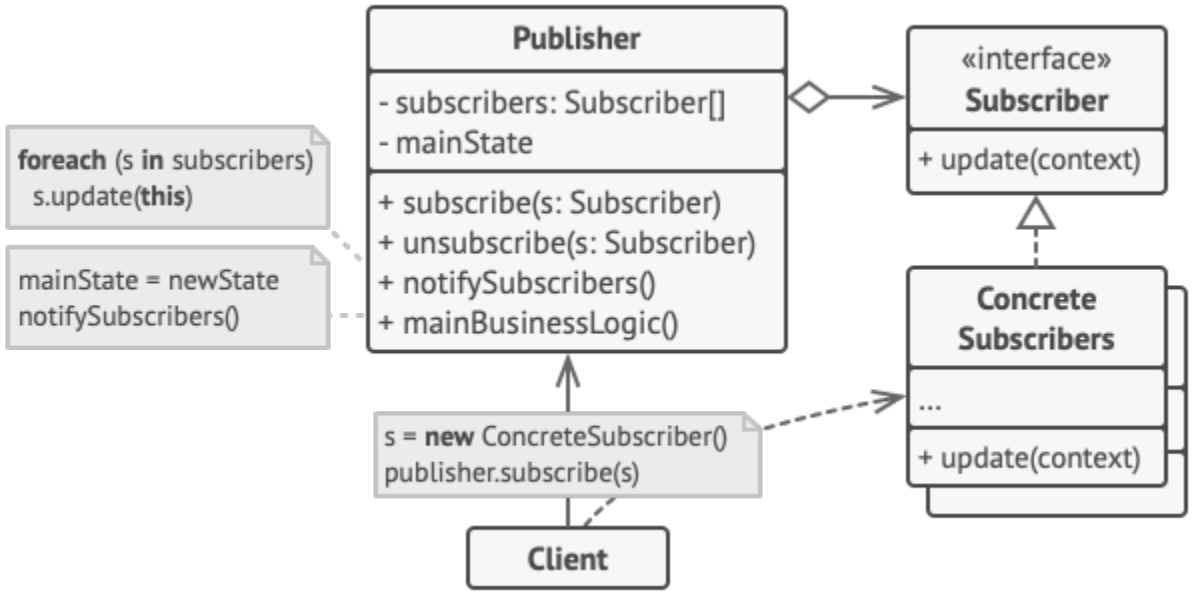
Propósito: Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él. Notifican cambios de estado y monitorizan el estado interno de objetos.



Observer es un caso particular del patrón publish/subscribe

Patrón Publish-Subscribe

- **Propósito**: remitentes envían mensajes no programados a suscriptores. Este patrón es parte del middleware orientado a mensajes (MOM) y utilizado en sistemas distribuídos. Atraviesa redes.
- **Ventajas**: Scalability, loosely coupling
- **Posibles usos**: subscripción a noticias

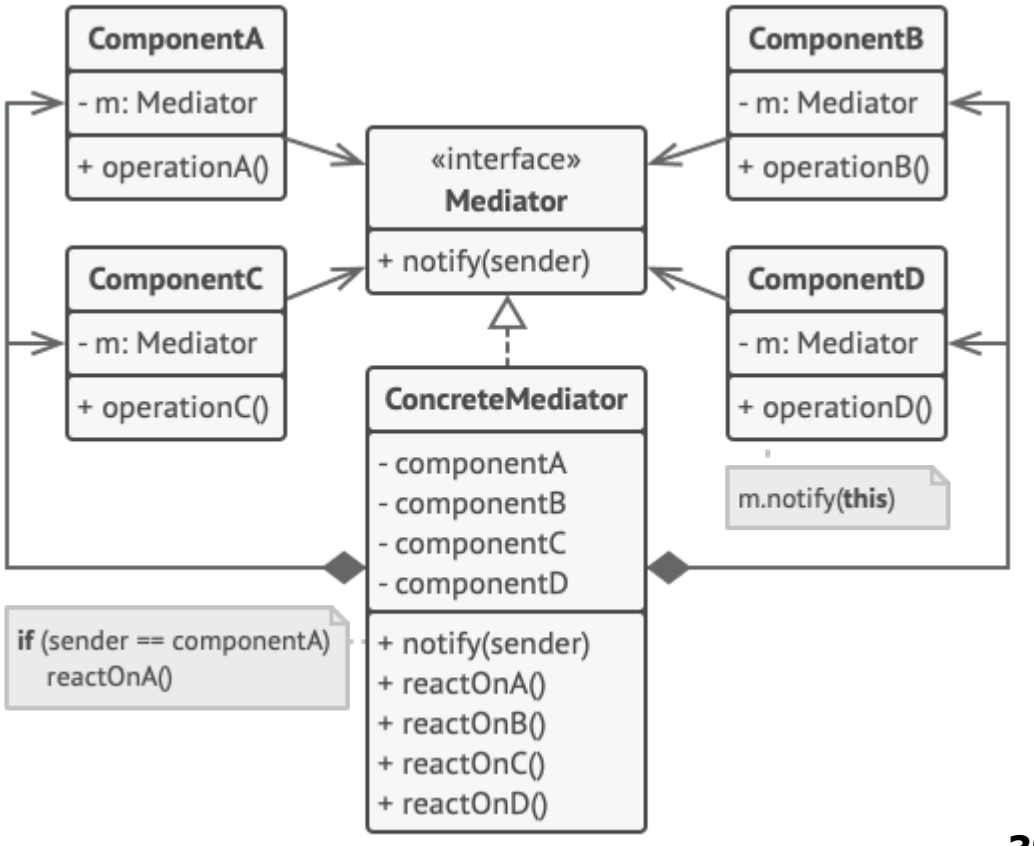


Patrón Mediator

- **Propósito**: encapsula la comunicación entre objetos a través de un objeto denominado “mediator”.
- **Ventajas**: Reduce el acoplamiento y dependencias entre objetos haciendo que se comuniquen indirectamente a través del elemento “mediator”. Es útil durante tareas de mantenimiento y refactorización del código.

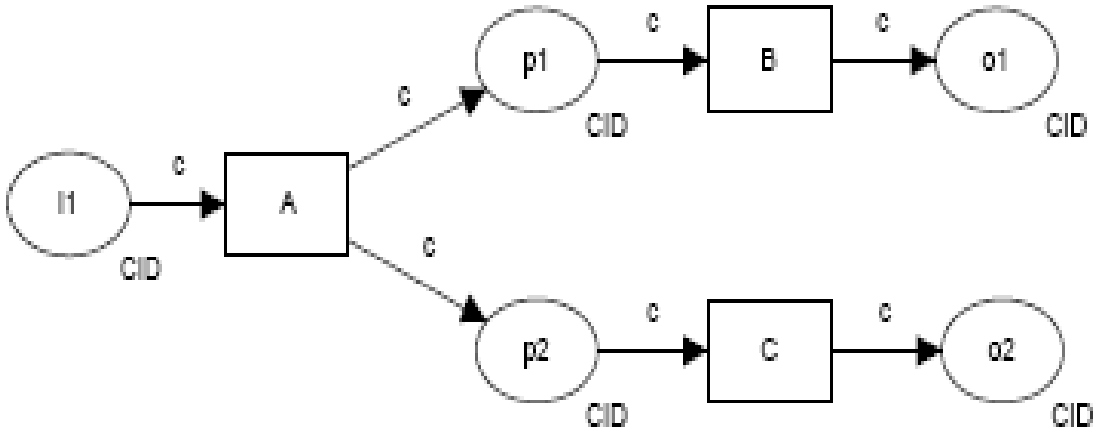
Los componentes definen la lógica de negocio y una referencia a la interfaz del mediator

La interfaz del mediator define formas de comunicación entre los componentes



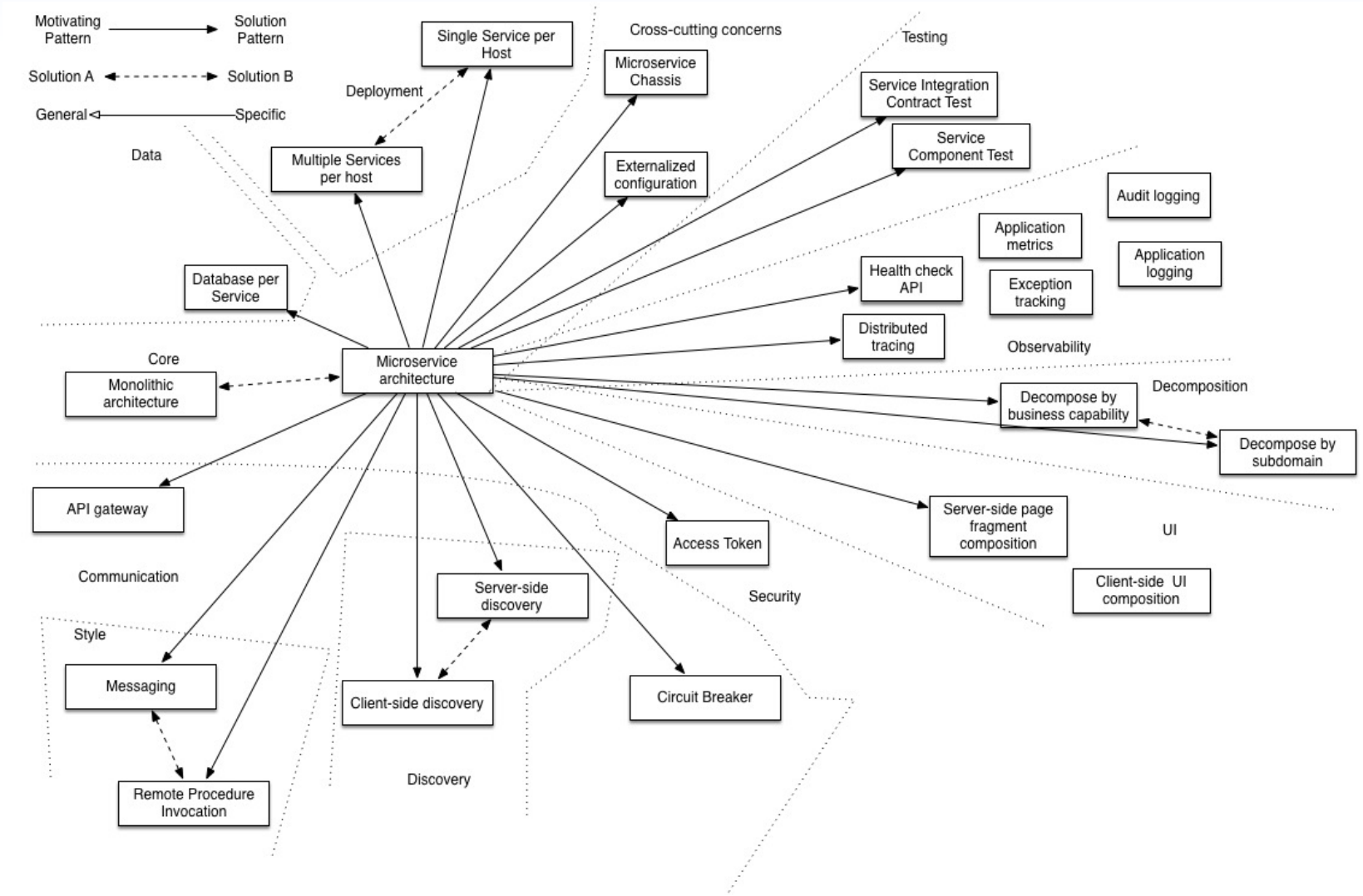
Otros Patrones: Parallel Split

- **Propósito**: Es un cierto punto del flujo donde un thread individual se divide en varios threads que pueden ejecutarse en paralelo permitiendo la ejecución simultánea de varias actividades. Uso potencial en orquestración de servicios Web (BPEL). Es un patrón básico de control de flujo



Muy utilizado en reservas de viajes u hoteles vía Web para hacer consultas paralelas a diversas bases de datos

Arquitecturas de Micro-servicios (MSA)



Patrones de Micro Servicios

Patrones de descomposición: (i) por capacidades de negocio, (ii) por subdominio

Base de datos por servicio

API Gateway

Descubrimiento de cliente y de servidor

Invocación remota de mensajes

Circuit Breaker (time-out para peticiones web)

Access Token

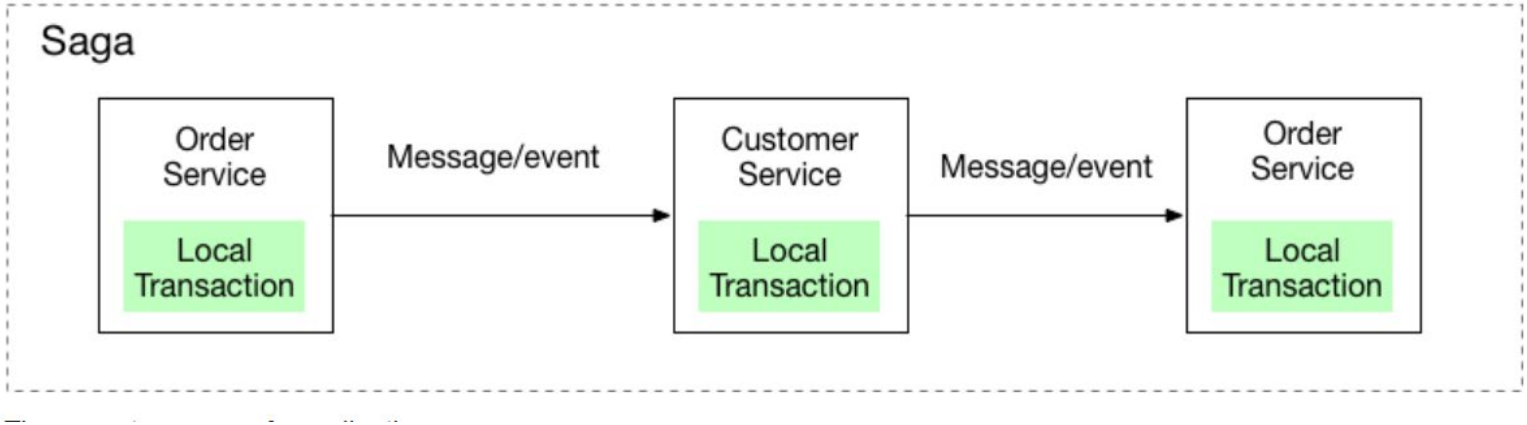
SAGA

CQRS (Command Query Responsibiity Segregation)

Patrones de Micro Servicios: SAGA

Problema: Implementar transacciones que abarque varios microservicios

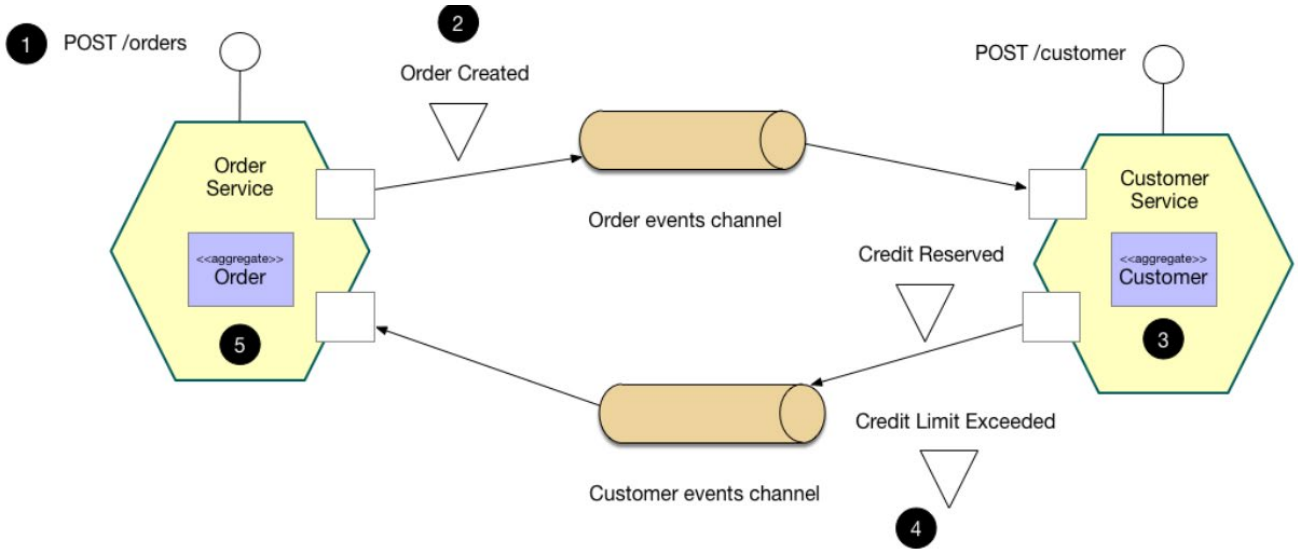
Solución: Implementar cada transacción de negocio con SAGA que es una secuencia de transacciones donde cada una de ellas actualiza la base de datos y publica un mensaje para la siguiente transacción SAGA. Si una transacción falla, SAGA ejecuta una compensación deshaciendo los cambios



Patrones de Micro Servicios: SAGA

Coordinación mediante coreografía: cada transacción local publica eventos que disparan las transacciones en otros servicios

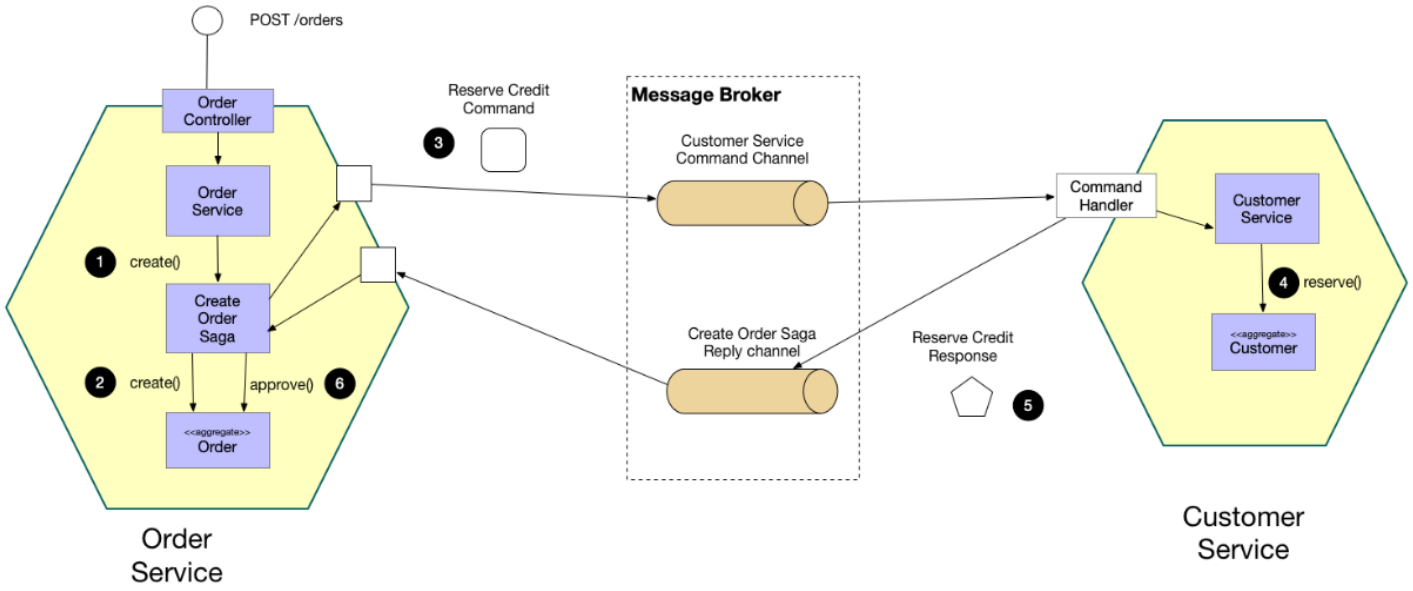
1. The `Order Service` receives the `POST /orders` request and creates an `Order` in a `PENDING` state
2. It then emits an `Order Created` event
3. The `Customer Service`'s event handler attempts to reserve credit
4. It then emits an event indicating the outcome
5. The `OrderService`'s event handler either approves or rejects the `Order`



Patrones de Micro Servicios: SAGA

Coordinación mediante orquestación: existe un objeto que "orquesta" y dice a los participantes que transacción local se debe ejecutar

1. The `Order Service` receives the `POST /orders` request and creates the `Create Order` saga orchestrator
2. The saga orchestrator creates an `Order` in the `PENDING` state
3. It then sends a `Reserve Credit` command to the `Customer Service`
4. The `Customer Service` attempts to reserve credit
5. It then sends back a reply message indicating the outcome
6. The saga orchestrator either approves or rejects the `Order`

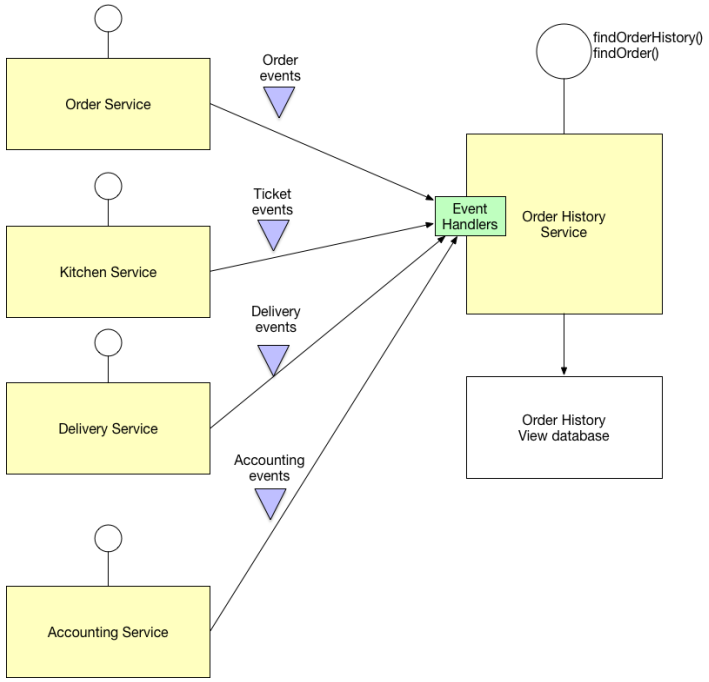


Patrones de Micro Servicios: CGRS (Command Query Responsibility Segregation)

Problema: Como implementar una query que recibe datos de múltiples servicios

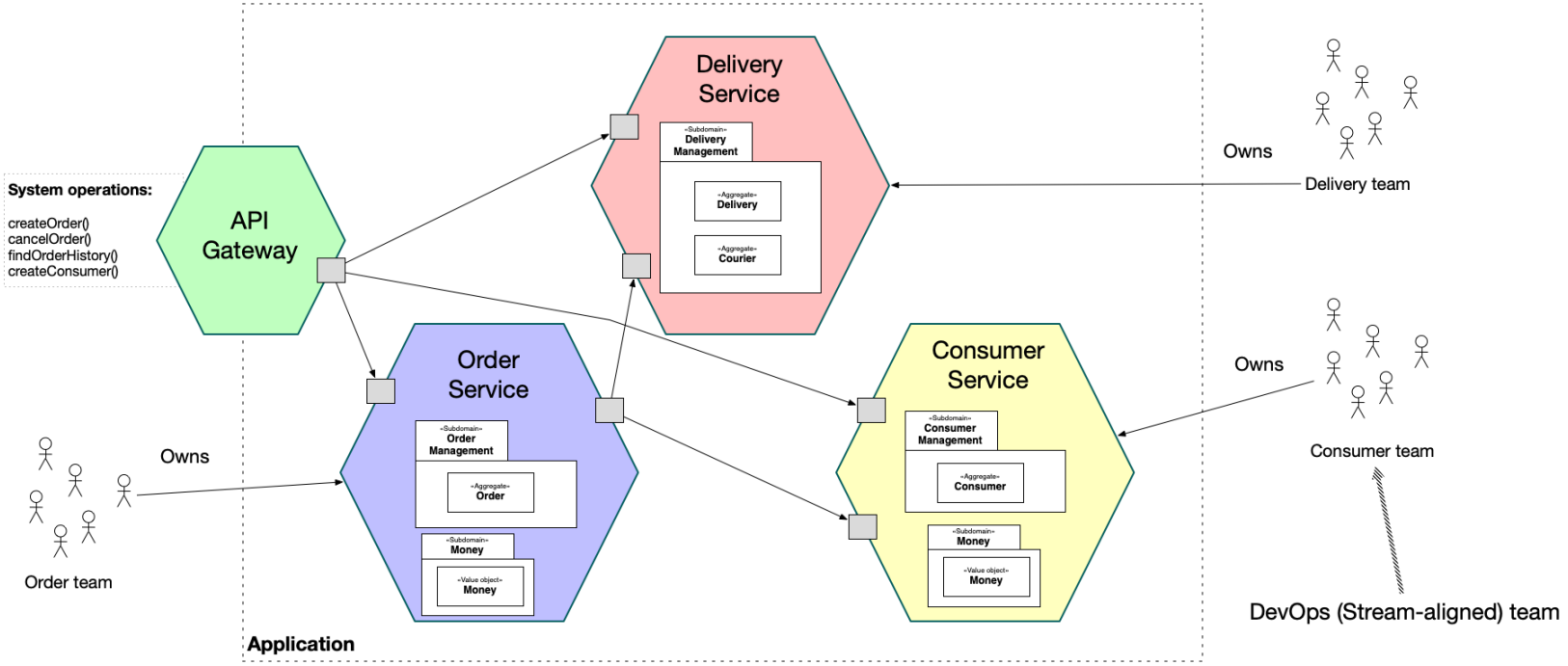
Solución: Diseñar una vista de la base de datos que es solo una réplica que soporta una o varias “queries”. La aplicación mantiene la base de datos actualizada mediante suscripción a eventos de dominio publicados por cada servicio propietario de los datos. Normalmente son bases de datos NoSQL.

Eventuate (eventuate.io) es una plataforma de gestión de datos distribuida para desarrollo de microservicios centrada en la lógica de negocio con soporte para CQRS



Ejemplos de Arquitecturas

Arquitectura de microservicios

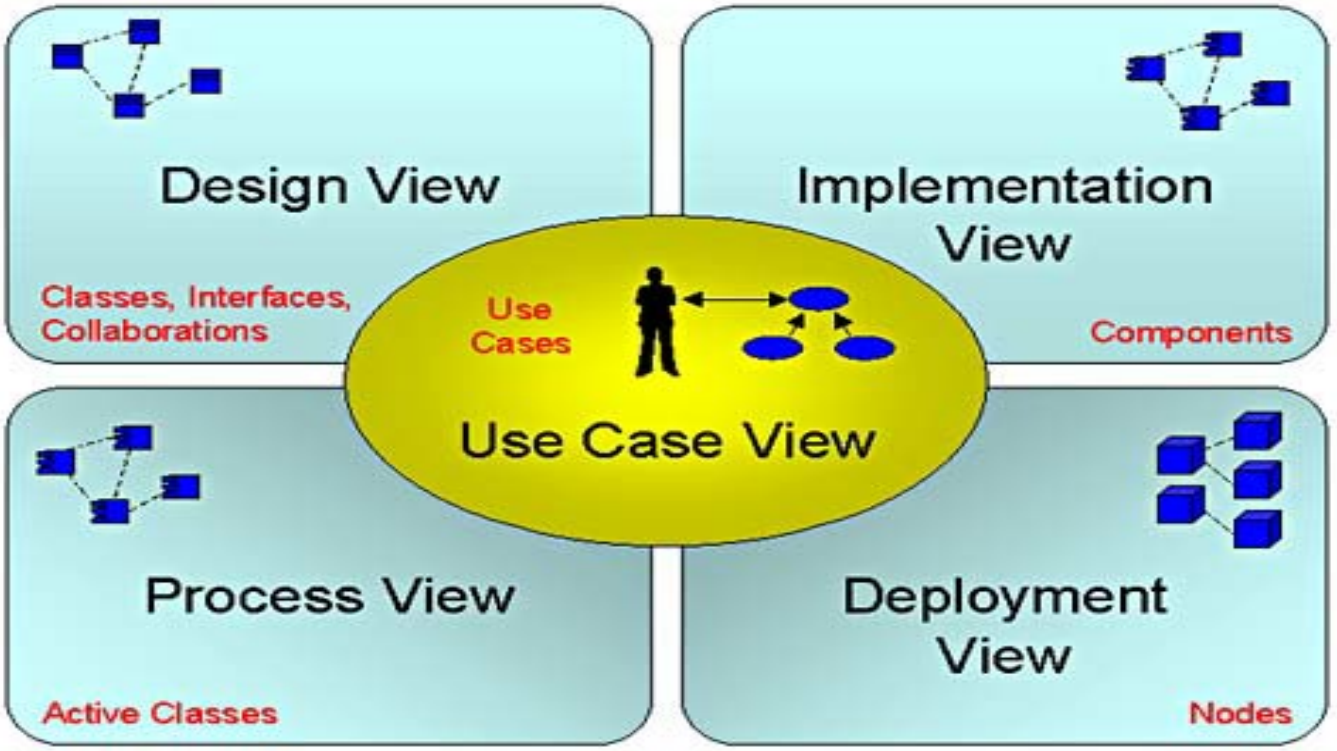


Vistas Arquitectónicas

- Permiten describir el sistema y la arquitectura desde diferentes puntos de vista o perspectiva
- Cada vista representa los intereses de un grupo de usuarios
- Existen diferentes modelos de vistas (Kruchten, Soni, Woods & Rozanski)
- Cada vista se representa mediante un conjunto de diagramas UML

Vistas Arquitectónicas

Modelo de vistas “4+1” (Kruchten, 1995)



Fue el primer modelo de vistas arquitectónicas propuesto y que luego se ha utilizado en todas las herramientas UML para tener diferentes tipos de diagramas. Una vista representa la perspectiva de un usuario en la arquitectura.

Vistas Arquitectónicas

- **Vista lógica**: captura las entidades lógicas (software) de un sistema y cómo se interconectan.
- **Vista física**: captura las entidades físicas (hardware) de un sistema y sus interrelaciones.
- **Vista de despliegue**: captura cómo las entidades lógicas se reparten en entidades físicas.
- **Vista de comportamiento**: captura el comportamiento esperado del sistema o partes de éste.
- **Vista de casos de uso**: captura los requisitos funcionales que serán ofrecidos por el sistema de información.

Arquitecturas y Diseño UML

- Vistas y Documentación:
  - Las arquitecturas deben documentarse como un conjunto de vistas arquitectónicas.
  - Utilizar solamente aquellas vistas necesarias o de interés para los “stakeholders”.
  - Uso de UML 2.5.x con herramientas de modelado UML. **NO Word ni Visio si utilizas UML!!!**
  - Decidir de antemano que diagramas son necesarios para describir la arquitectura de nuestro sistema (complejo).

Arquitecturas y Diseño UML

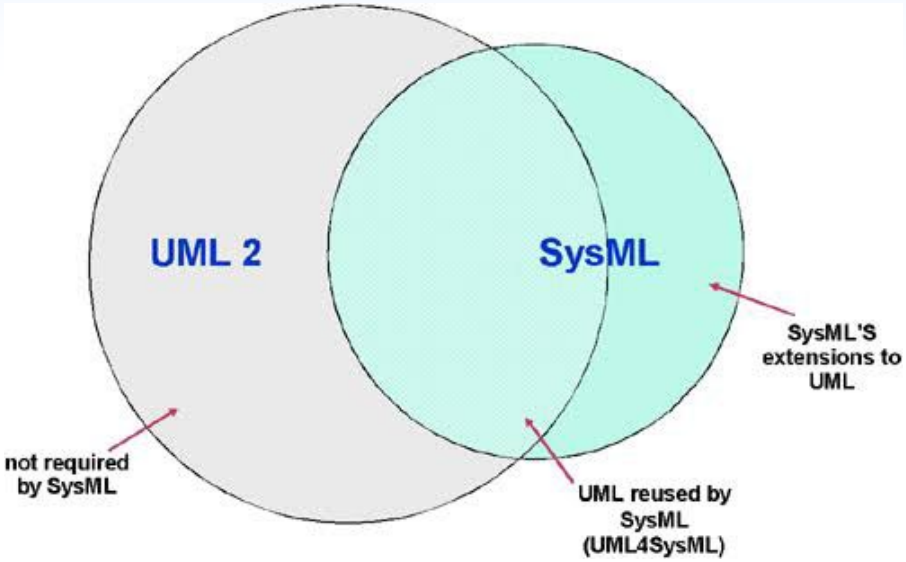
- Más de 9 tipos de diagramas UML para representar las distintas vistas arquitectónicas.
- **No es necesario utilizar todos los diagramas UML para describir una arquitectura software**.
- Existen perfiles UML como RM-ODP o SysML para sistemas distribuidos o para describir la ingeniería de sistemas. Un perfil de UML se instala como un plug-in de una herramienta y proporciona una librería gráfica de símbolos
- Uso de frameworks propios (TOGAF, Zachman, DODAF, MODAF, RM-ODP, GERAM, etc.) para describir arquitecturas de sistemas distribuidos, empresariales o militares.

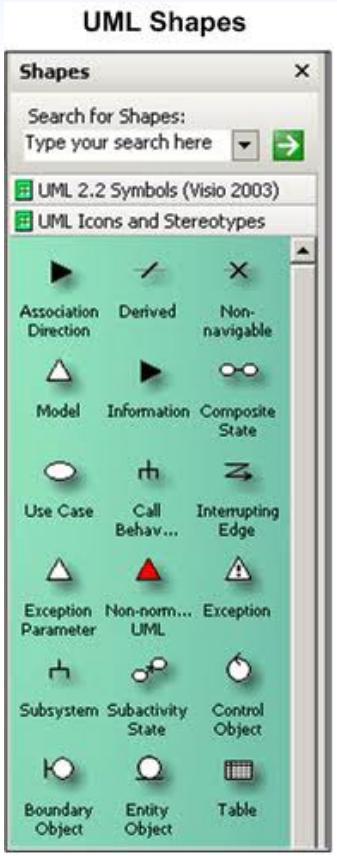
Diseño con UML

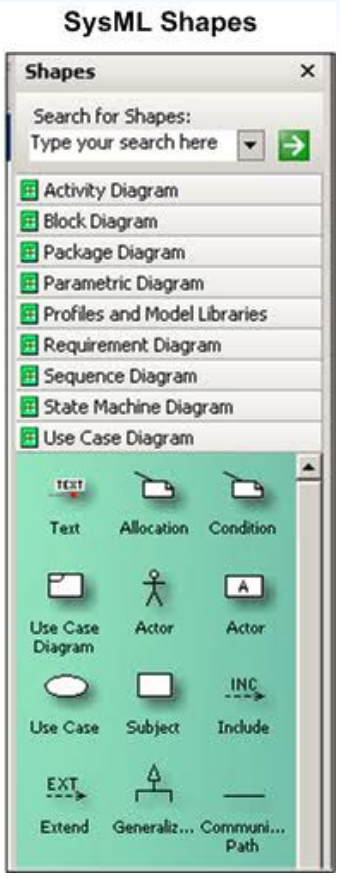
- Es una notación que incluye diferentes tipos de diagramas para modelar y diseñar sistemas software.
- Permite modelar vistas arquitectónicas a través de los diferentes diagramas UML
- Captura la información acerca de la estructura estática y el comportamiento dinámico de un sistema
- Dispone de un lenguaje de restricciones (OCL) para especificar pre-condiciones y post-condiciones
- Es generalista, pero para algunos dominios dispone de “perfiles” específicos (e.g., SysML, MARTE)

Para describir la arquitectura software de un sistema no es necesario utilizar todos los diagramas UML. Solamente aquellos que sean necesarios. Para ello es necesario conocer en que vistas arquitectónicas estamos interesados. Por ejemplo, al cliente final solo le interesaría la vista de casos de uso mientras que al encargado de desplegar los servidores solo le interesa la vista de despliegue.

Diseño con UML: SysML

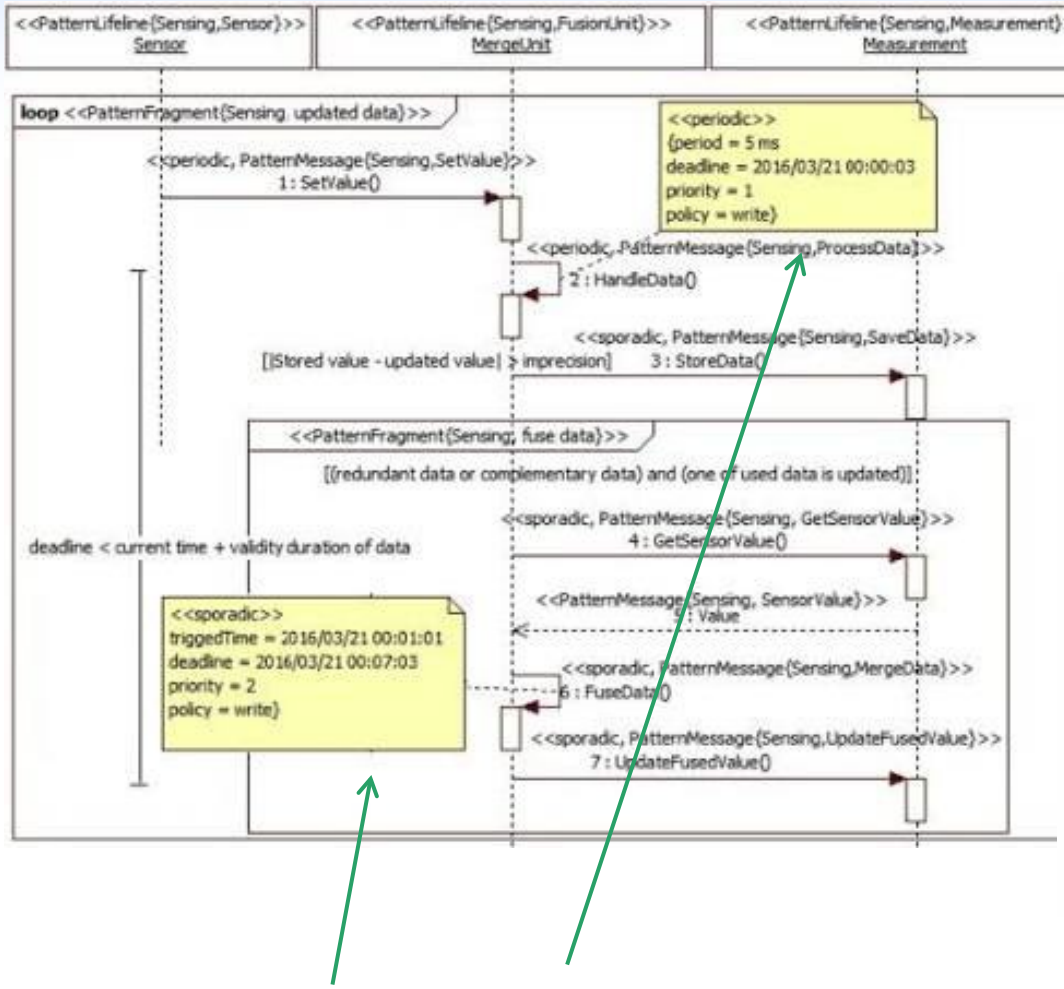


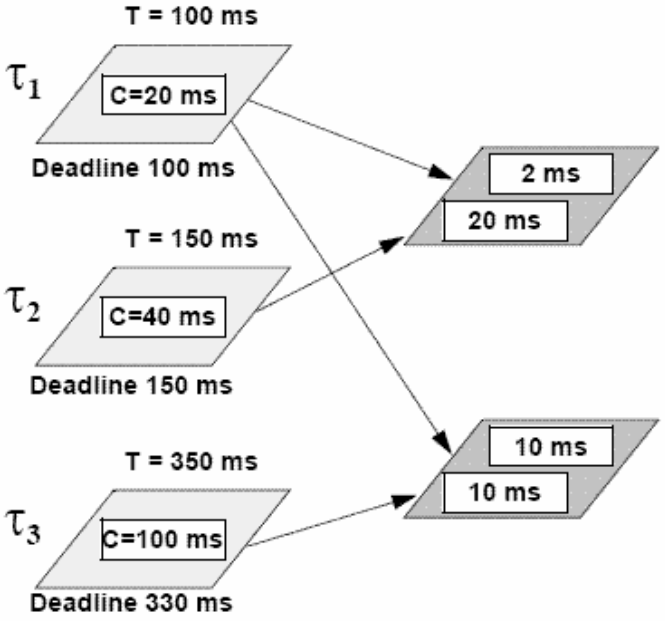




Extensión de símbolos a la paleta del UML tradicional

Diseño con UML: MARTE





Extensiones de tiempo real

Vistas y Diagramas con UML

- **Vista Estática (lógica)**
  - Diagrama de clases, diagrama de objetos
- **Vista de Casos de uso**
  - Diagrama de casos de uso
- **Vista de Despliegue**
  - Diagrama de despliegue
- **Vista de Proceso**
  - Diagrama de secuencias, diagramas de estado
- **Vista de Implementación**
  - Diagrama de componentes

Vistas y Diagramas con UML

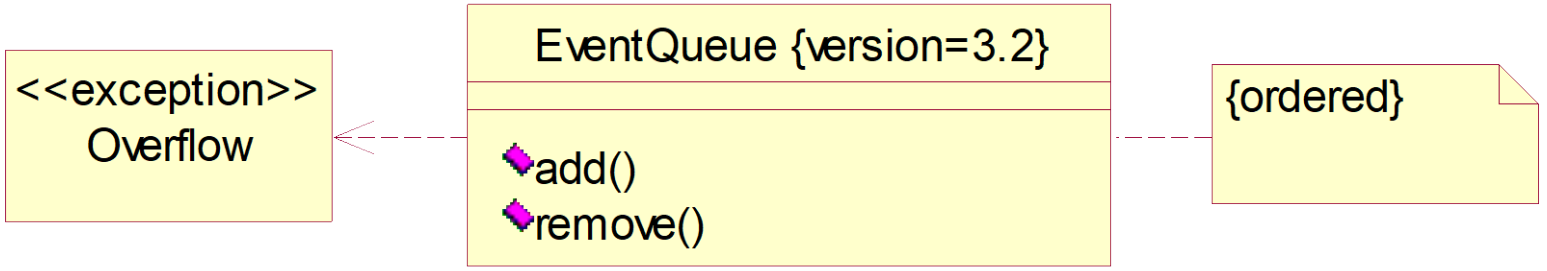
Area	Vista	Diagramas	Principales Conceptos
Estructural	Vista Estática	Diagrama de Clases	Clase, asociación, generalización, dependencia, realización, interface.
	Vista de Casos de Uso	Diagrama de Casos de Uso	Caso de uso, actor, asociación, extención, inclusión, generalización.
	Vista de Implementación	Diagrama de Componentes	Componente, interface, dependencia, realización.
	Vista de Despliegue	Diagrama de Despliegue	Nodo, componente, dependencia, localización.
Dinámica	Vista de Máquina de Estados	Diagramas de Estados (ciclo de vida de objeto)	Estado, evento, transición, acción.
	Vista de Actividades	Diagrama de Actividades	Estado, actividad, transición, concurrencia (fork), reunión (join).
	Vista de Interacción	Diagrama de Secuencia	Interacción, objeto, mensaje, activación.
		Diagrama de Colaboración	Colaboración, interacción, rol de colaboración, mensaje.
Gestión del Modelo	Vista de Gestión del Modelo	Diagrama de clases	Paquete, subsistema, modelo.
Extensibilidad	Todos	Todos	Restricciones, estereotipos, valores etiquetados.

UML: Comportamiento Dinámico

- Vista de Interacción: modela como interactúan los objetos para realizar una funcionalidad del sistema
  - Diagrama de secuencias
- Vista de Máquina de estados: modela el ciclo de vida de una instancia de una clase en estados y transiciones.
  - Diagrama de transición de estados
- Vista de Actividades: modela flujos de trabajo (workflows)
  - Diagrama de Actividades

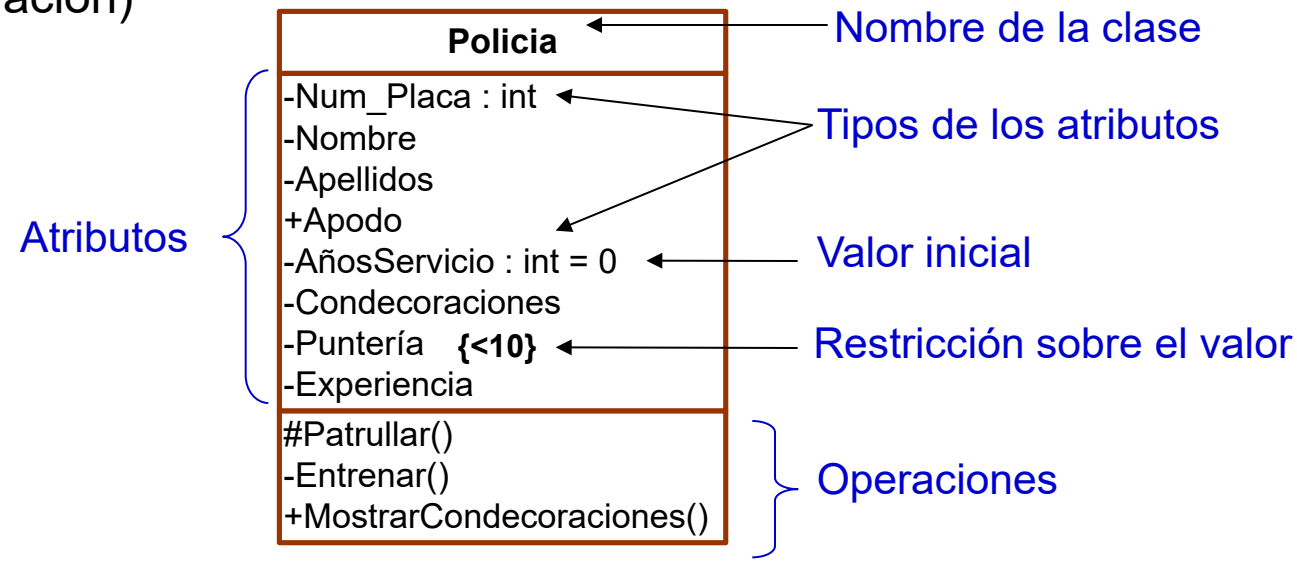
UML: Extensiones

- **Estereotipos**:
  - Extienden la semántica del elemento sobre el que se aplica
  - Permite representar una variación de un elemento existente que posee otra intención, o distinción de uso
  - Se expresan generalmente (entre << y >>)
- **Valores etiquetados**
  - Extiende las propiedades de un elemento de UML, permitiendo añadir nueva información en la especificación del elemento
  - Cadenas con el nombre de la etiqueta, un signo igual y un valor
- **Restricciones**
  - Notación formal con OCL



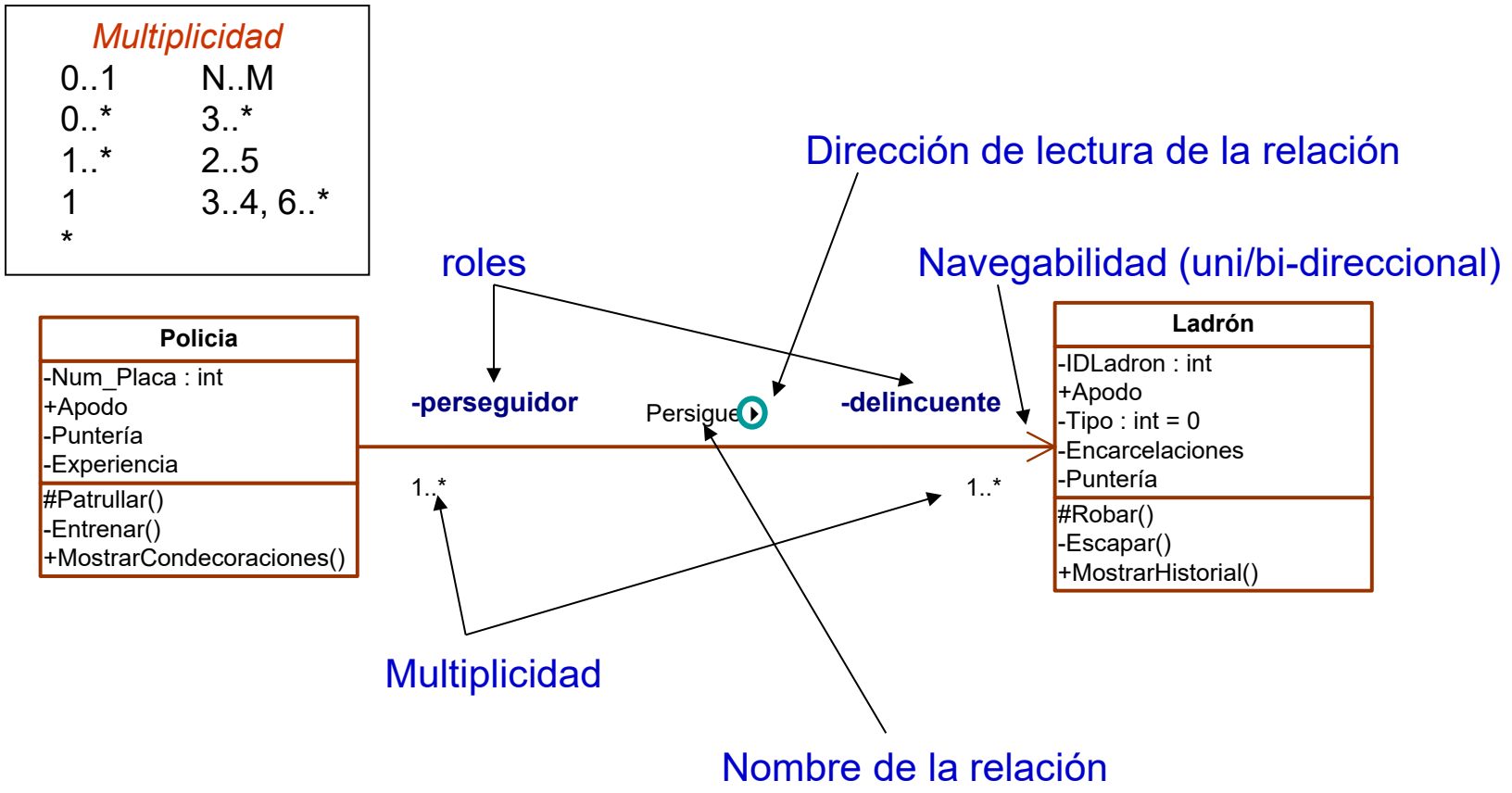
UML: Diagrama de Clases

- Representan la vista estática del sistema y describen conceptos (generalmente) del mundo real
- Nombre clase (entidad), atributos, métodos
  - **(-) Privado**: Los atributos/operaciones son visibles solo desde la propia clase.
  - **(#)** Los atributos/operaciones **protegidos** están visibles para la propia clase y para las clases derivadas de la original
  - **(+)** Los atributos/operaciones **públicos** son visibles a otras clases (cuando se trata de atributos se está transgrediendo el principio de encapsulación)



UML: Relación de Asociación

- Conexión semántica entre instancias de clases
- Proporciona una conexión para el envío de mensajes



UML: Relación de Agregación

- Lógica (agregación o composición débil): la parte puede pertenecer a varios agregados

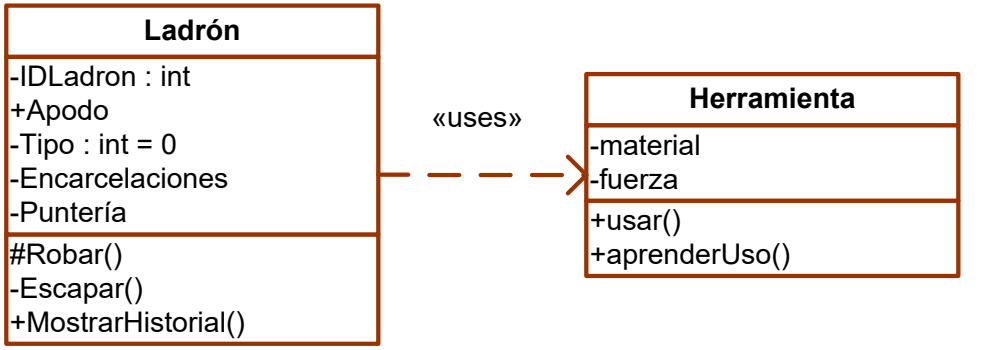


- Física (o composición fuerte): las partes sólo existen asociadas al compuesto (acceso a través de él)



UML: Relación de Dependencia

- Indica una relación semántica entre dos o más elementos del modelo en la cual un cambio al elemento proveedor puede requerir un cambio

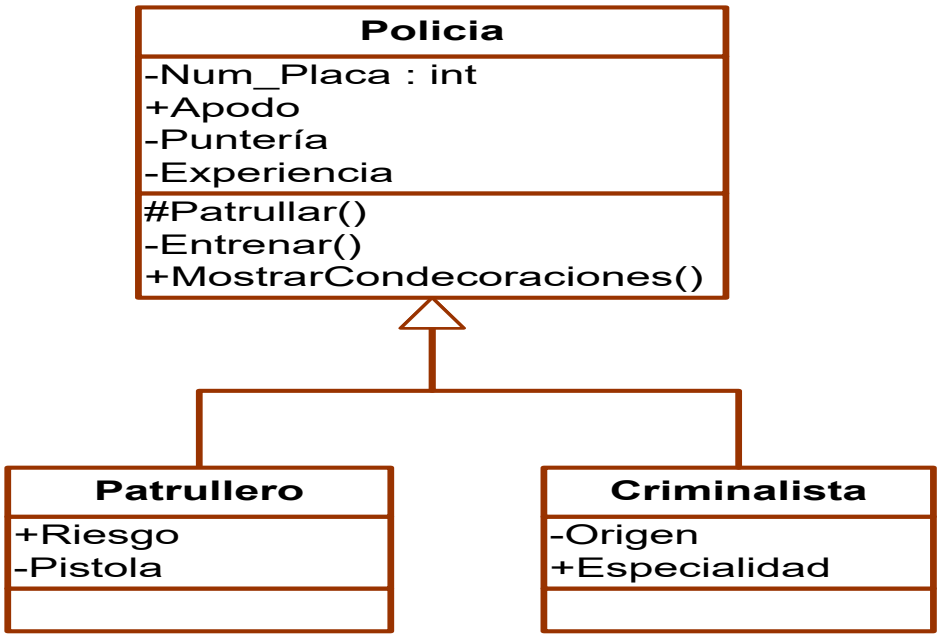


Mecanismos de extensión de UML

• Estereotipos	<<excepción>>
• Valores etiquetados	{versión=3.1}
• Restricción	{edad>18}

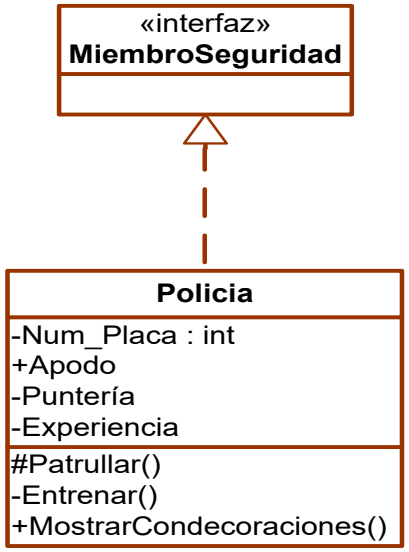
UML: Relación de Generalización

- Relación taxonómica entre una descripción general y otra más específica que la extiende
- Relación *"es un tipo de"*
- Representación del concepto de herencia



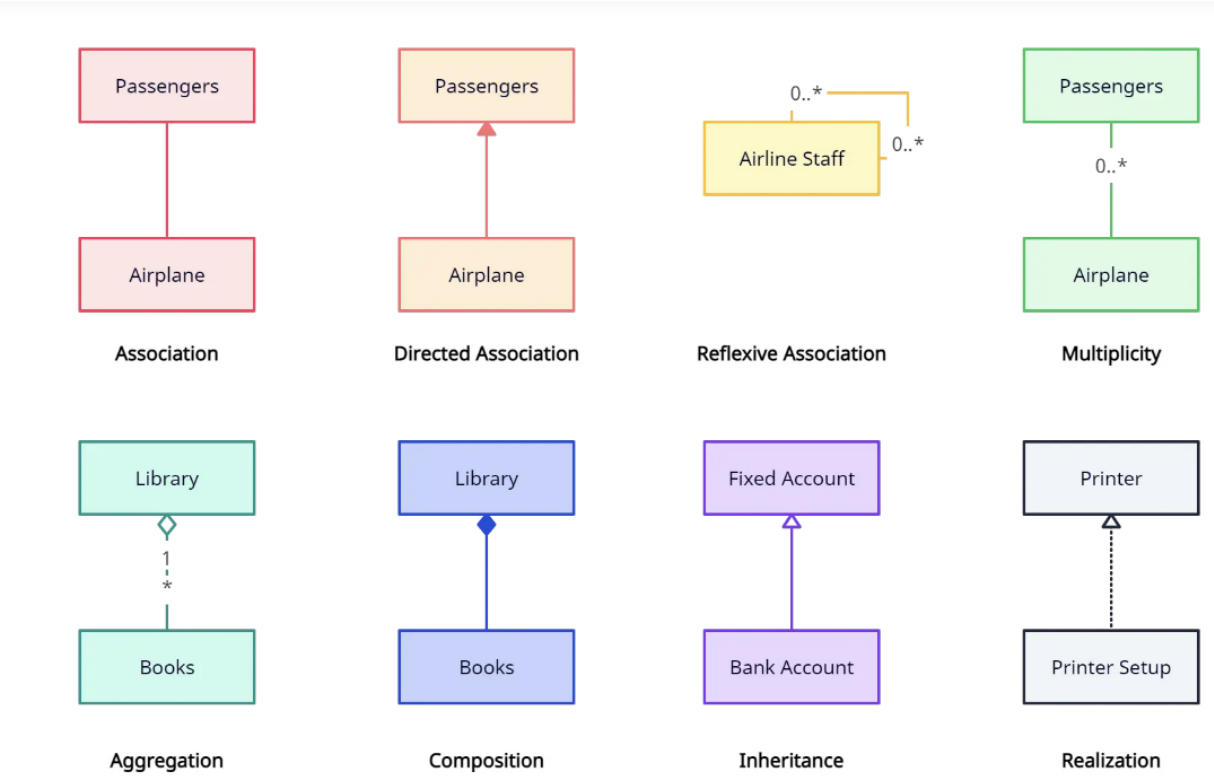
UML: Relación de Realización

- Es una relación que implica que la parte realizante cumple con una serie de especificaciones propuestas por la clase realizada
- Situaciones de aplicación:
  - Interfaces y las clases y componentes que lo implementan
  - Casos de uso y colaboraciones que lo realizan



UML: Relaciones

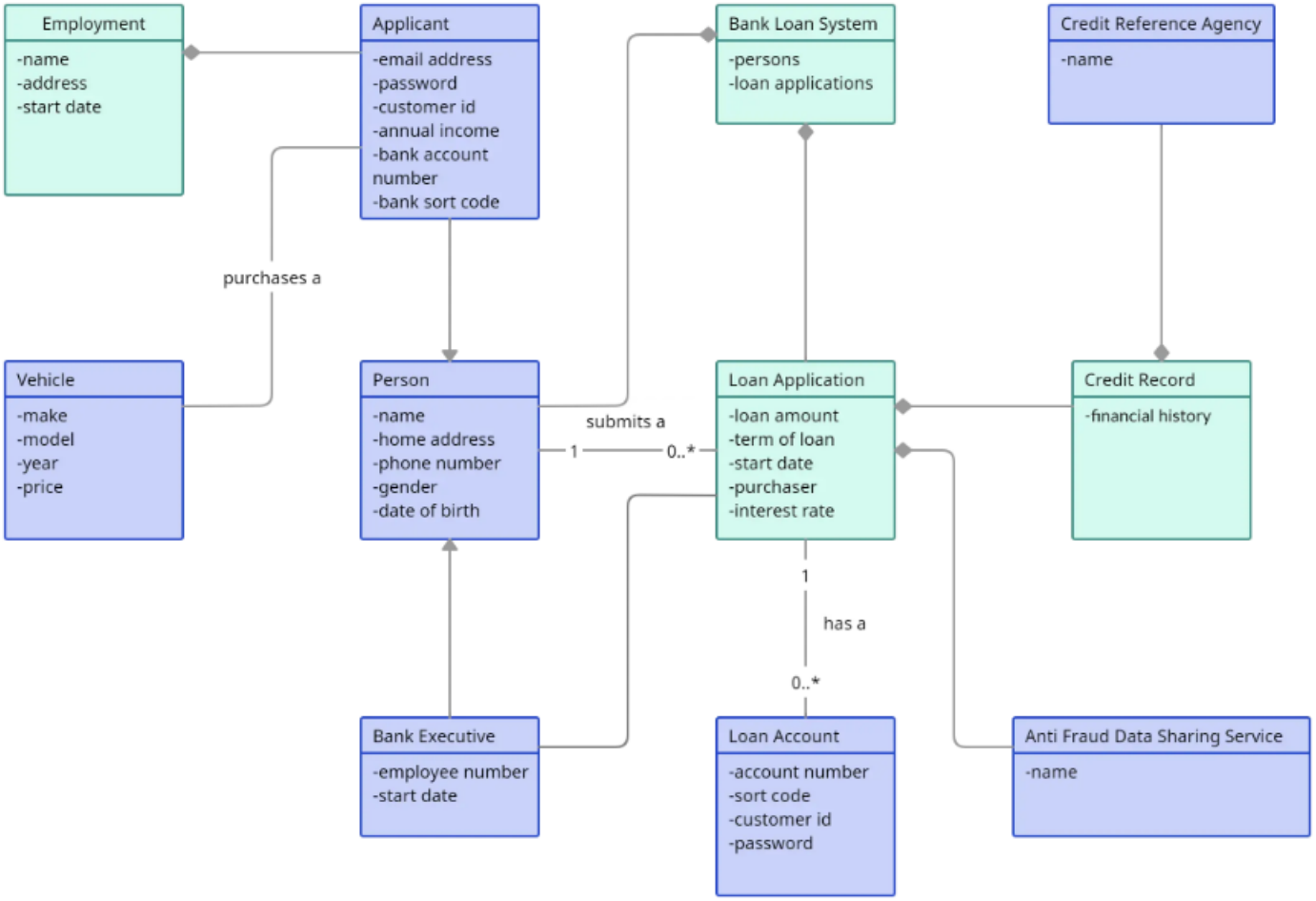
- Asociación, asociación dirigida, asociación reflexiva
- Agregación y composición
- Herencia
- Realización
- Otras: Uses, Import, etc



© Creately, 2024

UML: Diagrama de clases

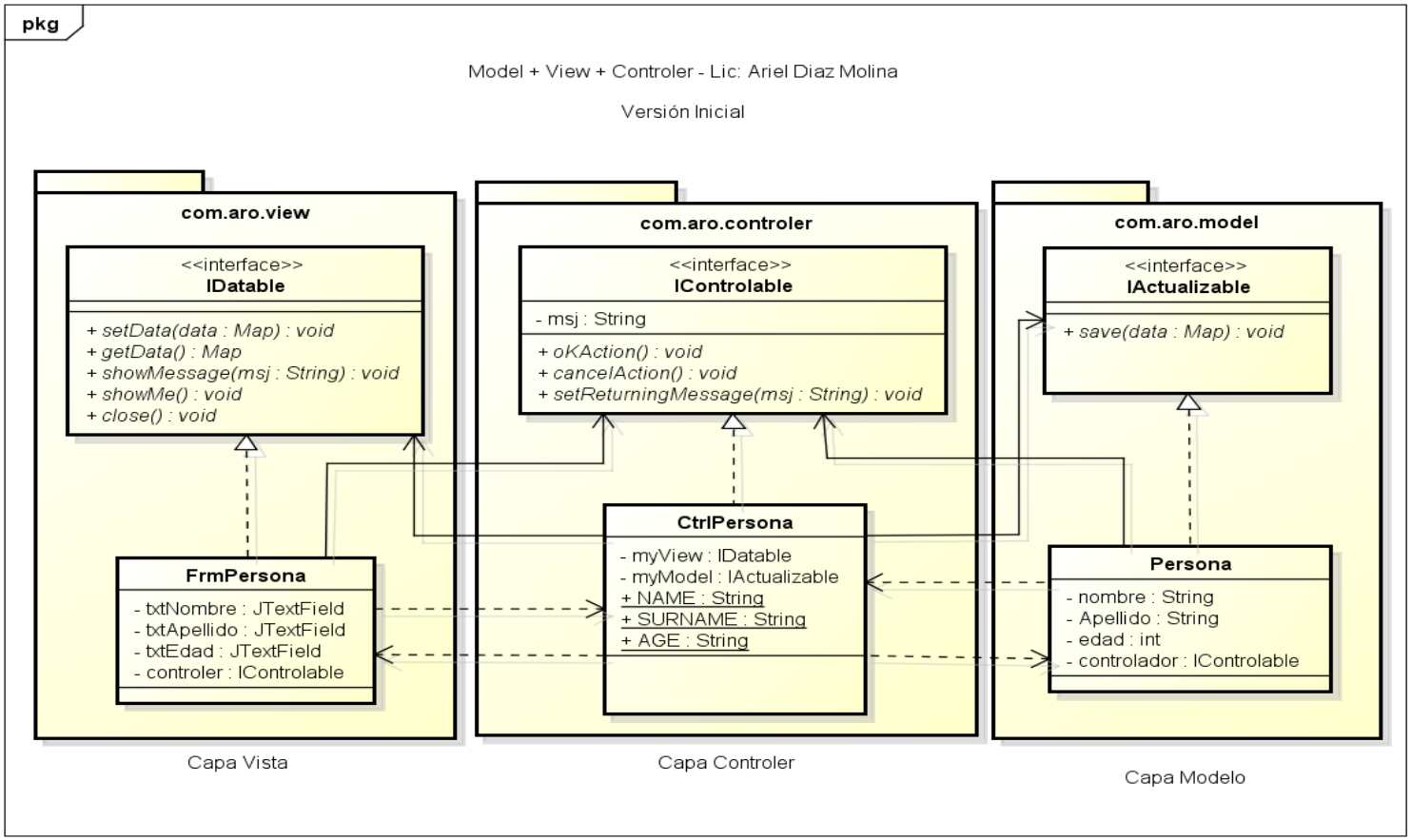
- Diferentes relaciones UML



© Creately, 2024

UML: Diagramas de Clases y Paquetes

- Utilizar paquetes UML para dividir responsabilidades
- Los paquetes contienen clases y componentes UML



powered by astah*

UML: Diagrama de Componentes

- Modelan el empaquetado físico del sistema en unidades reutilizables denominadas “componentes”.
- Cada componente implementa una o más clases del diseño, definen interfaces e incluyen código fuente o ejecutable.
- Suelen utilizarse para hacer referencia a una librería externa.

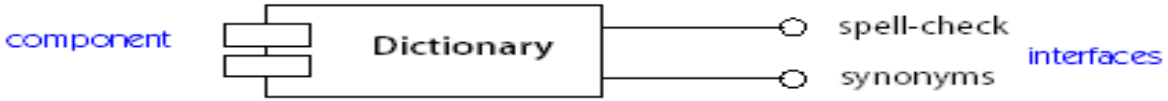


Figure 9-1. *Component with interfaces*

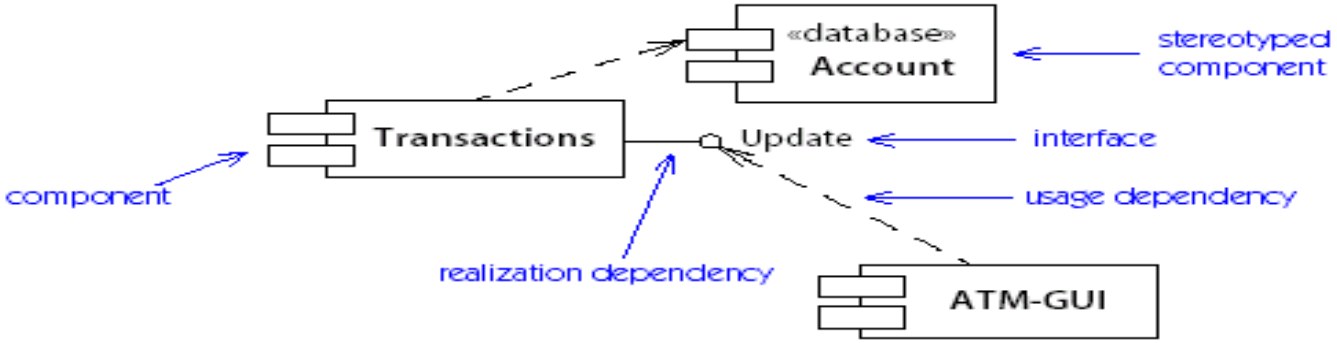
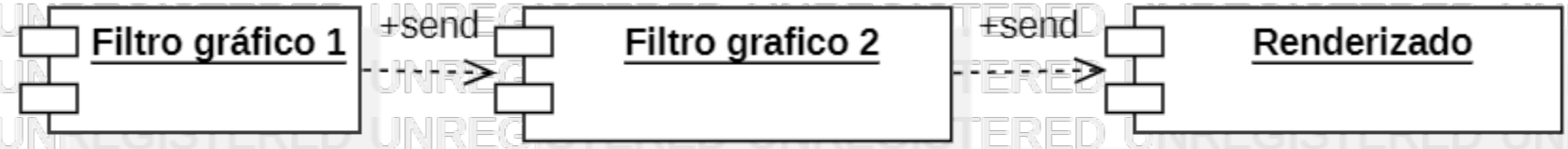


Figure 9-2. *Component diagram*

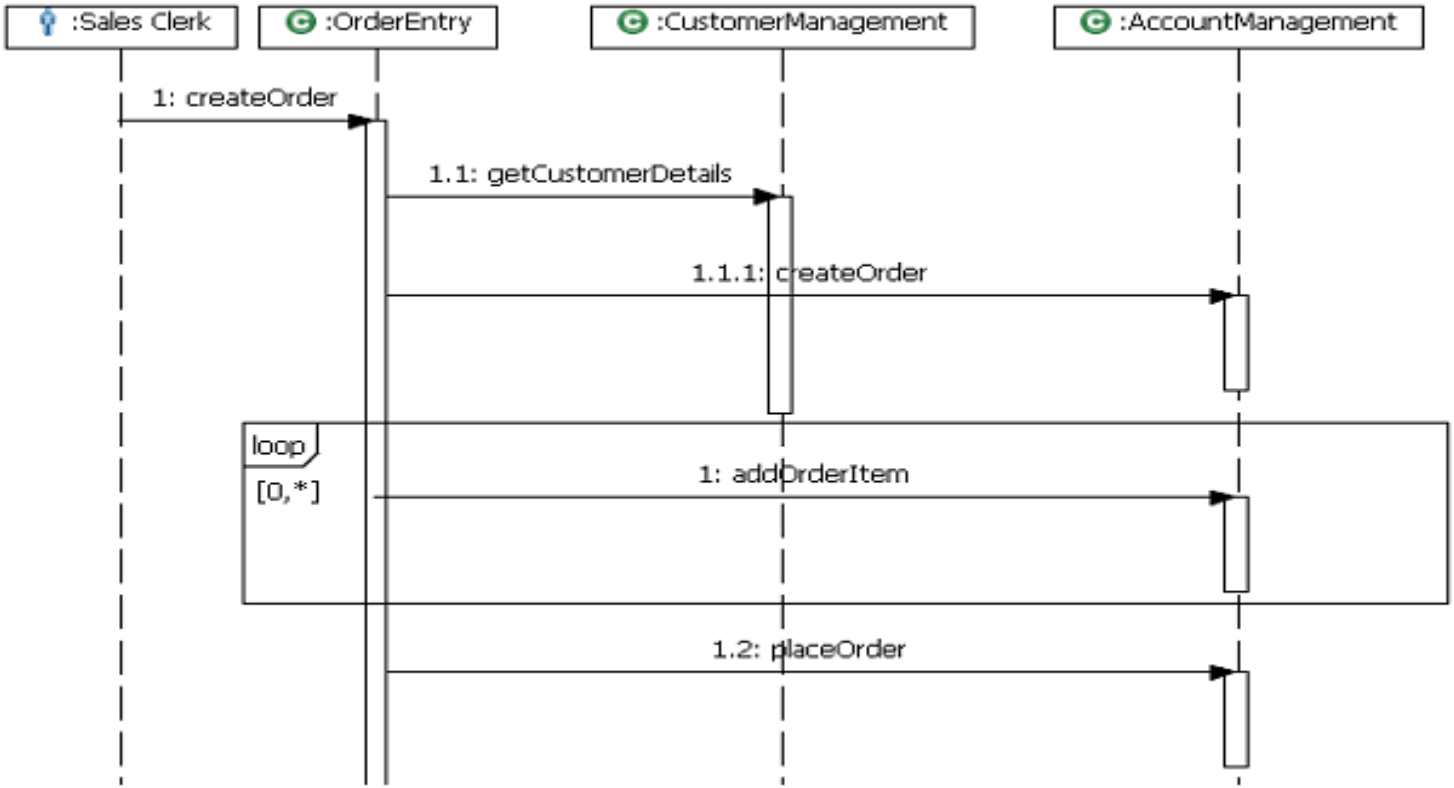
UML: Diagramas Componentes

- Estilo Pipe&Filter
- Filtros descritos usando componentes UML
- No serían necesarios los paquetes salvo en casos de gran complejidad
- Un componente puede invocar a otro si han sido desarrollados por la misma organización



UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED

UML: Diagrama de secuencias



UML: Diagrama de Despliegue

- Modela los nodos y unidades funcionales de un sistema así como la comunicación entre ellos.
- Se modela como la Vista de Despliegue

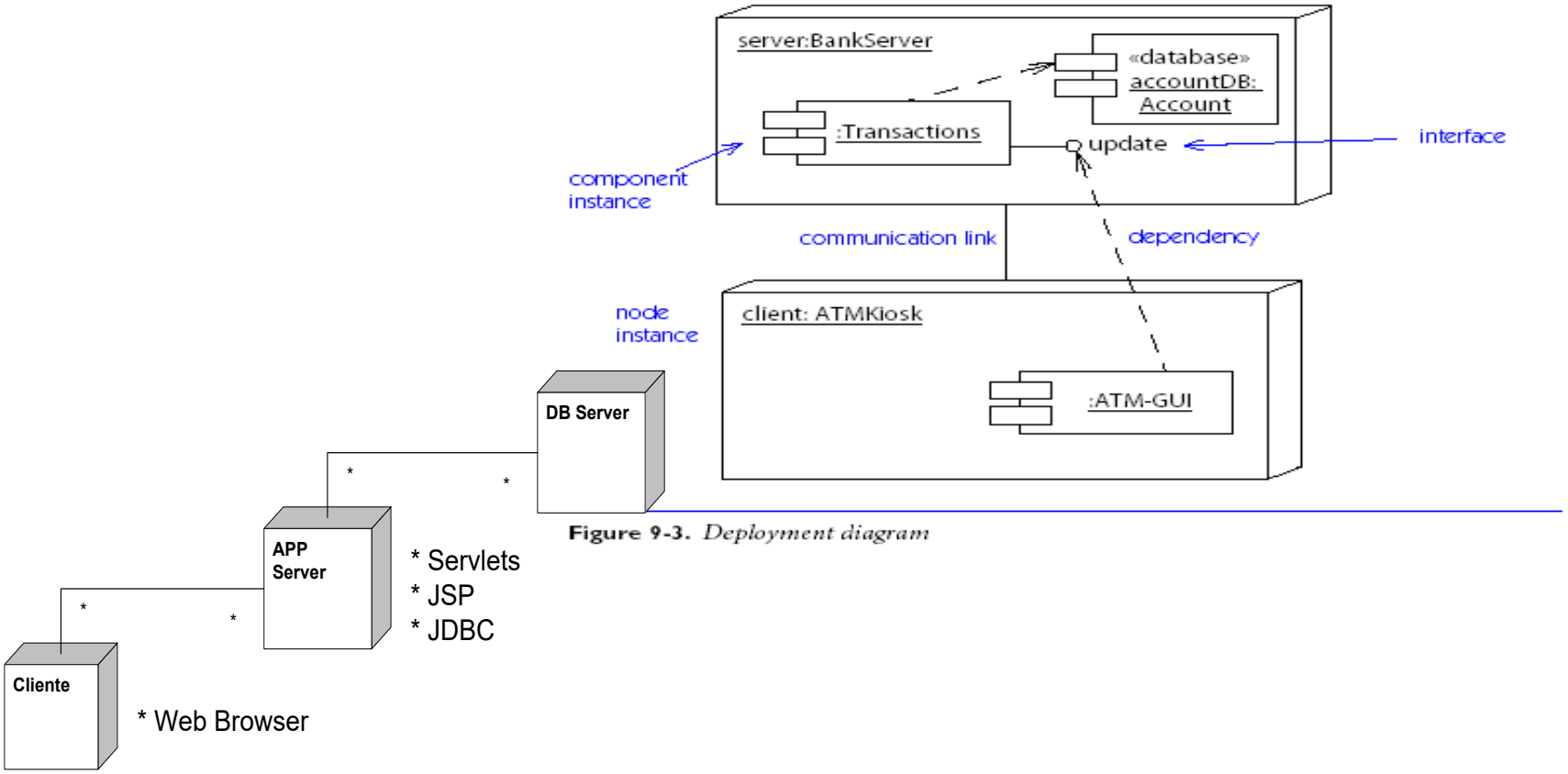
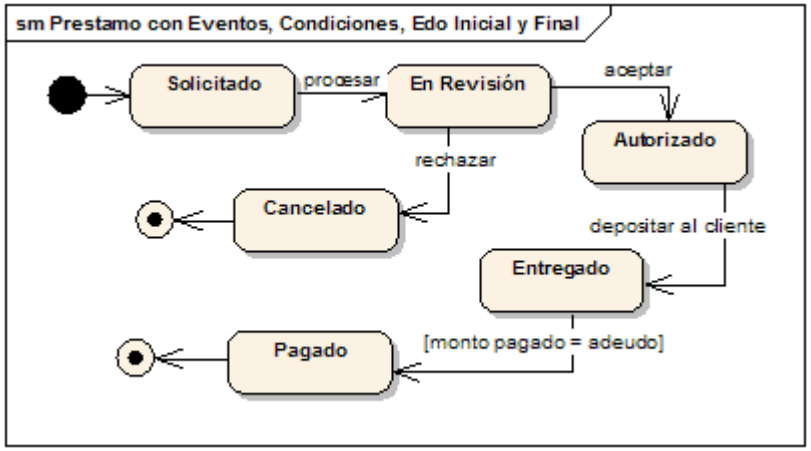
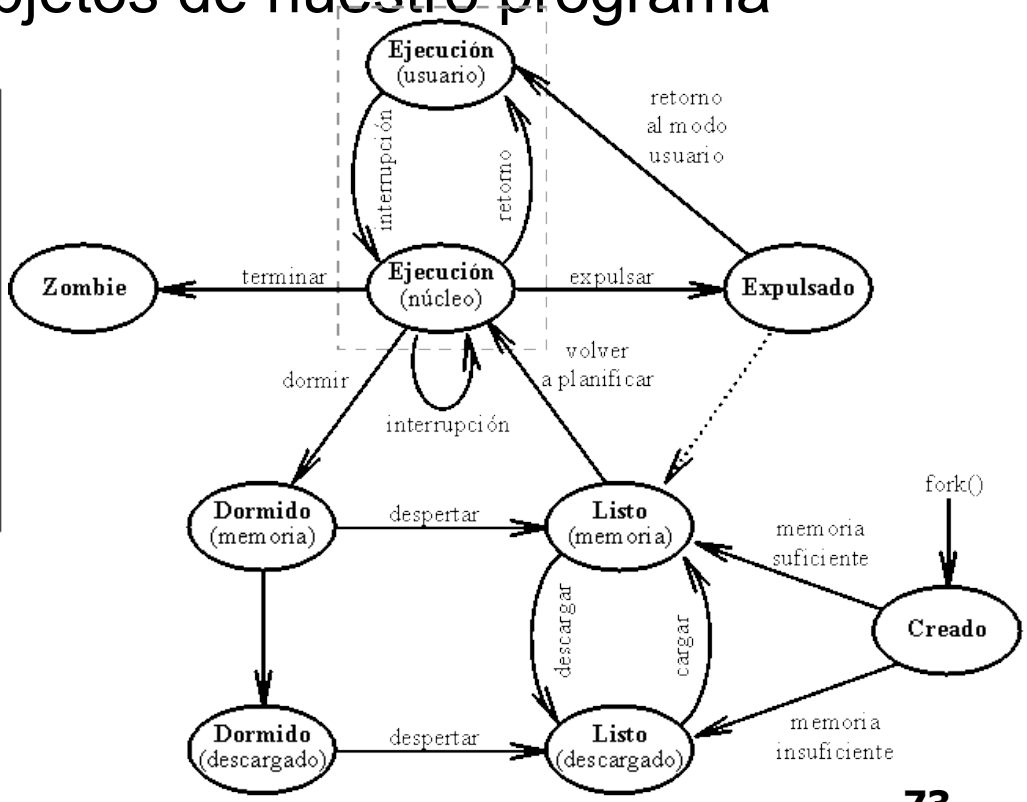


Figure 9-3. *Deployment diagram*

UML: Diagrama de Estados

- Definen estados y transiciones entre ellos motivados por eventos
- Invocaciones síncronas y asíncronas en el tiempo
- Se utiliza cuando estamos interesados en los estados y transiciones de los objetos de nuestro programa





Conclusiones

- Los patrones de diseño son soluciones codificadas de conocimiento reutilizables que han sido probadas.
- Los estilos arquitectónicos definen los grandes subsistemas ya áreas funcionales de la arquitectura software.
- Patrones y estilos se utilizan en la construcción de arquitecturas de referencia y software.
- UML ofrece una notación para describir y documentar arquitecturas (No es la única!) con diversos tipos de diagramas.
- Cada stakeholder tiene diferentes intereses y puntos de vista que se representan mediante diferentes diagramas.
- No es necesario utilizar todos los diagramas del UML, sólo los necesarios.

Enlaces

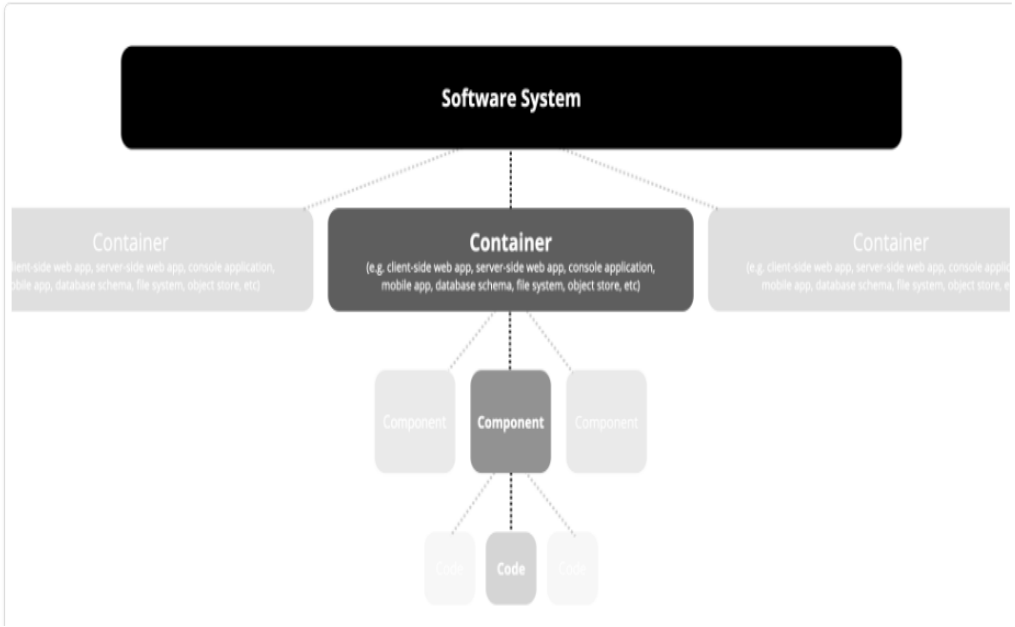
- Patrones de diseño Refactoring Guru: https://refactoring.guru/design-patterns
- Patrones de microservicios: https://microservices.io/patterns
- Diagramas UML: https://creately.com/blog/diagrams/uml-diagram-types-examples/
- Notación C4: https://c4model.com/

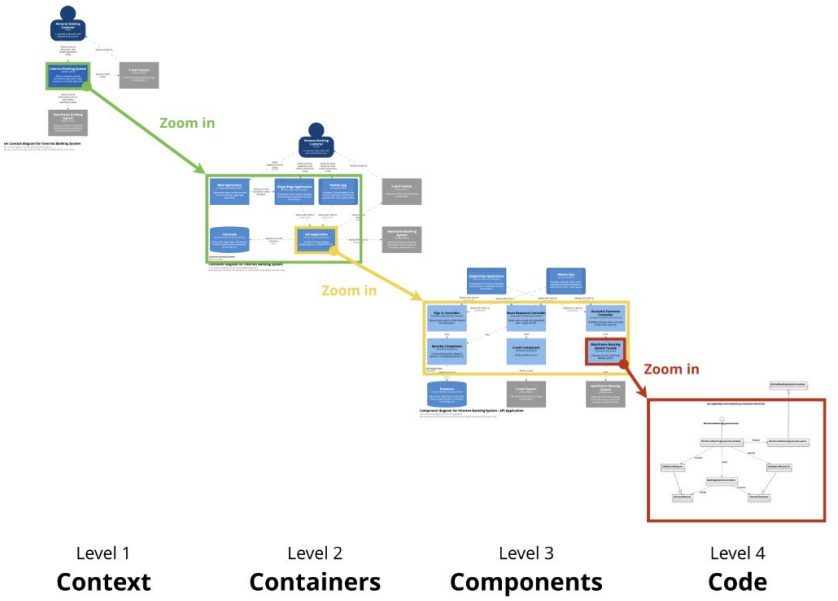
Notación C4

- Existen otras notaciones gráficas además de UML
- Creado para ayudar a los desarrolladores a comunicar la arquitectura y documentar el diseño del código
- Trata de reflejar como piensan los arquitectos cuando diseña el software
- C4 es fácil de aprender y utilizar
- C4 define 4 tipos de diagramas: **(i) System context diagram, (ii) Container diagram, (iii) Component diagram, (iv) Code diagram**
- **https://c4model.com/**

Notación C4: Abstracciones

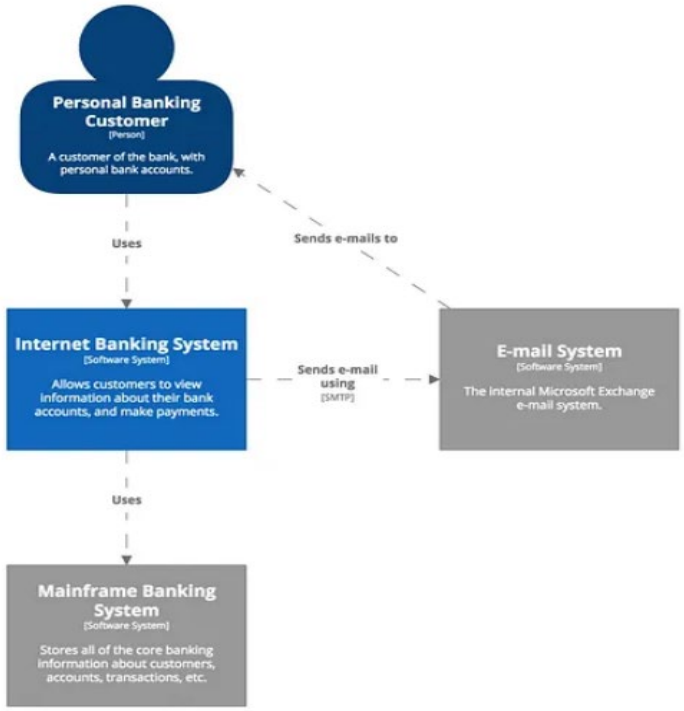
- C4 permite crear la estructura estática de un sistema
- Un sistema software se define en términos de “**containers**” que son aplicaciones y almacenes de datos
- Un “**container**” contiene varios componentes (“**components**”)
- Cada “**component**” se implementa mediante elementos de código (e.g. clases, objectos, funciones, etc.)





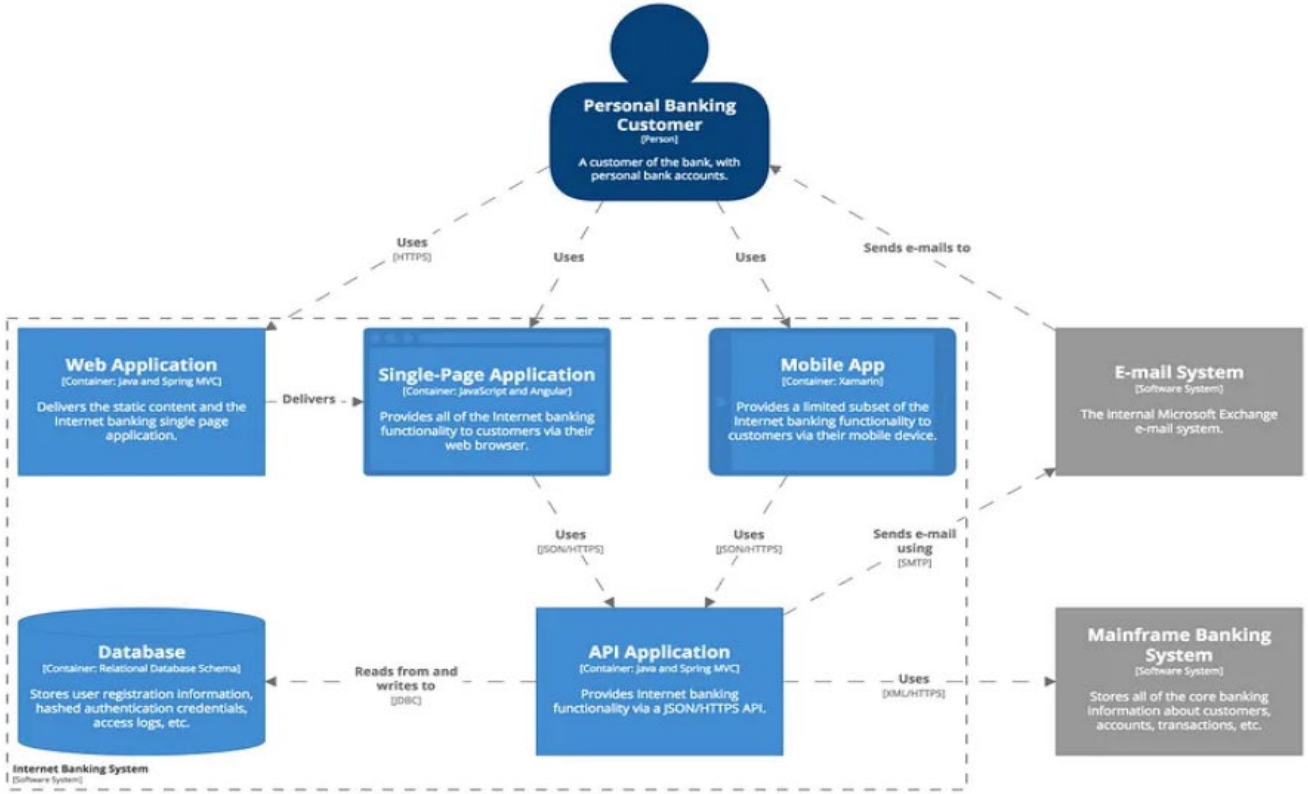
Notación C4: System context diagram

- Es el punto de partida para documentar el diseño de un sistema software
- No muestras los detalles internos de la arquitectura
- Se centra en los actores (“stakeholders”) involucrados y las actividades principales



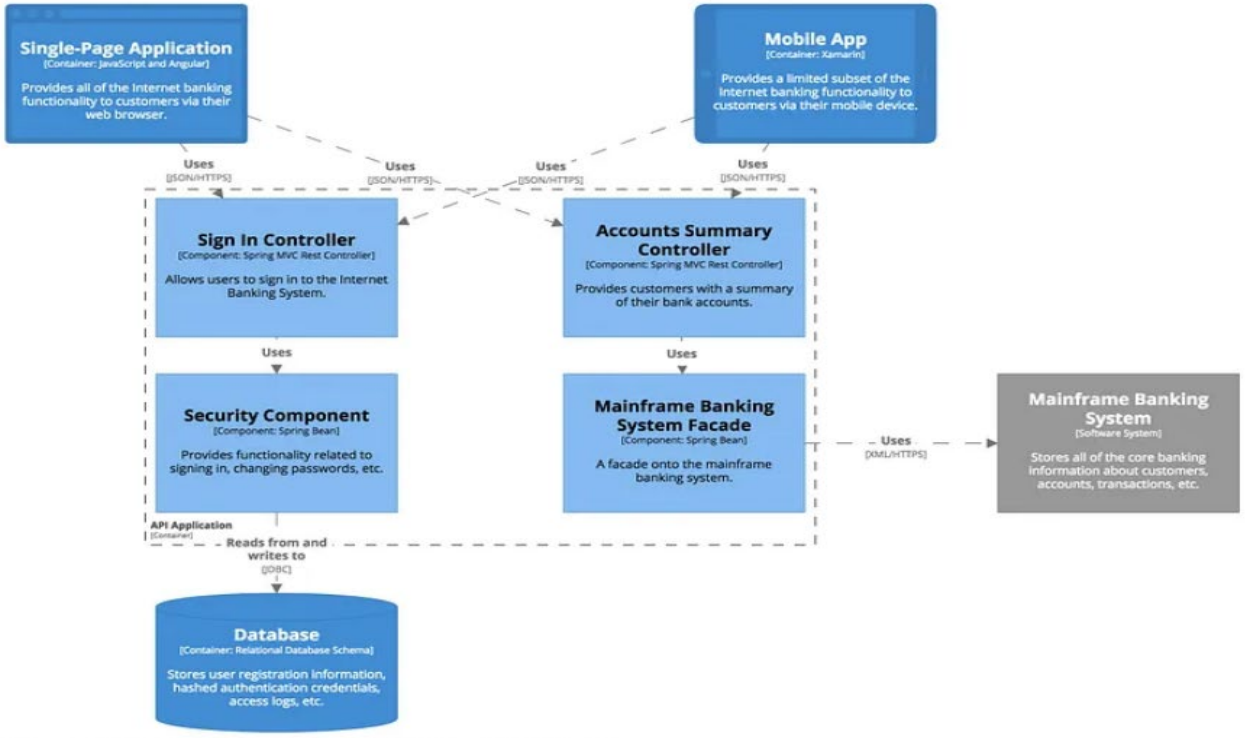
Notación C4: Container diagram

- Muestra los contenedores principales del sistema (aplicaciones, BBDD) refinando el diagrama de contexto
- Puede incluir decisiones tecnológicas



Notación C4: Component diagram

- Amplia el detalle de los contenedores y muestra los componentes del sistema
- Los componentes deben corresponder a abstracciones reales

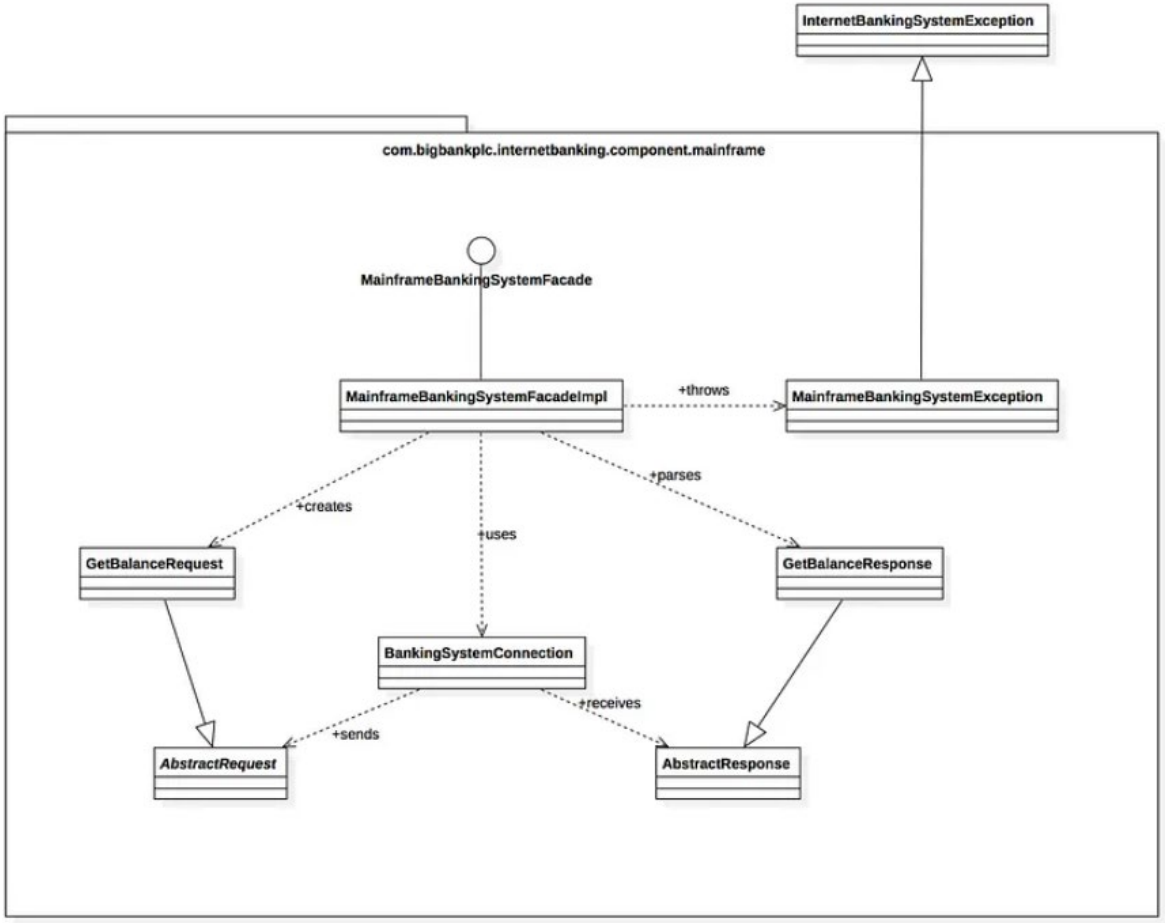


Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Last modified: Wednesday 02 May 2018 13:46 UTC

Notación C4: Code diagram

- Permite mostrar detalles de código de los componentes individuales, pero a nivel de diseño



Notación C4

- Existen también diagramas para mostrar los aspectos dinámicos y de despliegue basados en UML
- C4 no prescribe una notación específica, pero permite describir de manera simple y efectiva la arquitectura de un sistema
- Con C4 resulta más complejo describir estilos arquitectónicos
- Herramienta prototipo:
  - Archimate Modelling tool: https://github.com/archimatetool/archi